MLT LAB · MODEL PAPER

All 15 sets, solved.

Every question from the model paper with its code and the real output, for revising away from the laptop.

How to use this:

1. Read the **Guide** pages first: the 25 small Q1 tasks and the one Q2 pattern.
2. Then go set by set in this order: **1–4** → **6, 12, 13** → **5, 7, 15** → **14**.
3. For sets 8–11, read **Question 1**. Read Set 8's Question 2 only if you have time.

Tap a set on the contents page to jump to it.

Code is the same as the notebooks in `Model_Paper_Solved/`. Long outputs are cut after 16 lines.

Contents

★	Guide: which set is which · Q1 task bank · Q2 pattern · order · tips	
1	EDA + Simple Linear Regression	LEARN Q2
2	EDA + Multiple Linear Regression	LEARN Q2
3	EDA + Logistic Regression	LEARN Q2
4	EDA + Single Layer Perceptron	LEARN Q2
5	EDA + MLP Regression with Randomized / Grid Search	LEARN Q2
6	EDA + Random Forest (optimised)	LEARN Q2
7	EDA + MLP Classifier (optimised)	LEARN Q2
8	EDA + Agglomerative Clustering	Q2 IF TIME
9	EDA + Fuzzy C-Means	Q2 SKIP · learn Q1
10	EDA + SOM Clustering	Q2 SKIP · learn Q1
11	EDA + Apriori	Q2 SKIP · learn Q1
12	EDA + Random Forest (4 models)	LEARN Q2
13	EDA + AdaBoost (4 models)	LEARN Q2
14	EDA + Perceptron vs Logistic Regression	LEARN Q2
15	EDA + MLP Regression with Randomized Search	LEARN Q2

Guide

Every set from the model paper is solved in this PDF (and in the notebooks in `Model_Paper_Solved/`).

Which set is which

Set	Q1 dataset	Q2 asks for	Q2 is from	Learn Q2?
1	Diabetes	Simple linear regression	Exp 3	✅
2	Toyota	Multiple linear regression	Exp 3	✅
3	Stores*	Logistic regression	Exp 3	✅
4	Stores	Single layer perceptron	Exp 4	✅
5	Kasol*	MLP regression + Randomized/Grid search	Exp 5	✅
6	Toyota*	Random Forest + optimise	Exp 10	✅
7	IRIS	MLP classifier + optimise	Exp 5	✅
8	Diabetes	Agglomerative clustering + dendrogram	Exp 6	if time
9	IRIS	Fuzzy C-Means	Exp 7	❌ skip
10	Solar	SOM clustering	Exp 8	❌ skip
11	Diabetes	Apriori	Exp 9	❌ skip
12	Diabetes	Random Forest, 4 models	Exp 10	✅
13	Diabetes	AdaBoost, 4 models	Exp 10	✅
14	Diabetes	Perceptron vs logistic regression	Exp 4 + 3	✅
15	Solar	MLP regression + Randomized search	Exp 5	✅

Even for the ❌ sets, learn Q1. It's always the easy EDA question and carries its own marks.

* **Stand-in datasets.** We don't have every file the paper names, so these use a similar file. The code stays the same, and in the exam you change only the file name and the column names:

Paper asks for	Used here	Why it works
`Rampur.csv` (set 1)	`Kasol.csv`	Same river data, with the same `Wind` and `Discharge` columns

Paper asks for	Used here	Why it works
`Customers.csv` (set 3)	`Stores.csv`	Text columns + missing values above and below 10%
`Solar-Regression.csv` (sets 5, 15)	`Kasol.csv` → `Solar` column	A continuous solar radiation value to predict
`Precipitation.csv` (set 6)	`Toyota.csv`	Numbers + text columns, for the count plot and label encoding
`Supermarket.csv` (set 11)	`Shop1.csv`	Shopping baskets, the same format Apriori needs
`house_price.csv` (set 12)	`Diabetes.csv`	2 classes. For a price column, first do `(df["Price"] > df["Price"].median()).astype(int)`

`Diabetes.csv` is the standard Pima Indians diabetes data (with the `Outcome` column), and `IRIS.csv` is the standard Iris data.

How to learn this (the easy way)

Step 1: Q1 is only about 25 small tasks, repeated across the 15 sets

Learn each task **once** and you can answer every Q1. Here they are, with the sets that ask each one:

EDA (Exp 1A)

Task	Key code	Sets
Columns with nulls	`df.isnull().sum()`	1, 11
5-number summary + box plot	`df[col].describe()[["min","25%","50%","75%","max"]]` · `sns.boxplot(x=df[col])`	1, 9, 10, 11, 14
min / max / mean / mode	`pd.DataFrame({"min": num.min(), "max": num.max(), "mean": num.mean()})` (mode: `num.mode().iloc[0]`)	2, 4, 15
Count categorical / numerical columns	`df.select_dtypes("object")` · `df.select_dtypes("number")`	2
High +ve / high –ve / no correlation	`df.corr()`, then loop: > 0.5, < –0.5, close to 0	3, 5, 7
Pearson / Spearman heatmap	`sns.heatmap(num.corr(method="spearman"), annot=True)`	7

Task	Key code	Sets
Skewness: right / left / none	`num.skew()` : > 0.5 right, < –0.5 left · `num.plot(kind="kde", subplots=True)`	4, 10
Z-score outliers	`z = ((num - num.mean()) / num.std()).abs()` → `z > 3`	1, 6, 8, 12
Plots	`sns.catplot` (continuous vs category) · `sns.swarmplot` · bar: `value_counts().plot(kind="bar")` · `plt.scatter` · `sns.countplot(x=, hue=)` · `sns.pairplot(df, hue=)` · `sns.violinplot`	all

Pre-processing (Exp 1B)

Task	Key code	Sets
Fill missing (mean / median / mode) when % is above or below 10	`pct = df.isnull().mean() * 100` → `if pct[col] > 10: df[col] = df[col].fillna(df[col].mean())`	2, 3, 4, 5, 12
Nearest interpolation	`df[col].interpolate(method="nearest")`	8
Drop columns with too many missing	`df.drop(columns=pct[pct > 10].index)` · `df.dropna(axis=1)`	3, 5, 13
Label encoding	`LabelEncoder().fit_transform(df[col])`	6, 13, 15
One-hot encoding	`pd.get_dummies(df, columns=[col], dtype=int)`	7
StandardScaler	`StandardScaler().fit_transform(num)`	4, 10
Min-max (by hand)	`(x - x.min()) / (x.max() - x.min())`	9, 15
Z-score (by hand)	`(x - x.mean()) / x.std()`	10, 12
Binarize	`Binarizer(threshold=140).fit_transform(df[[col]])`	14

Feature selection (Exp 2)

Task	Key code	Sets
Drop low variance	`num.var()` → drop `< 10` · or `VarianceThreshold(0.3)`	6, 15
Drop highly correlated	upper triangle of `corr().abs()` , drop columns `> 0.5` (or 0.8)	14
SelectKBest / SelectPercentile	`SelectKBest(f_classif, k=4).fit(X, y).get_feature_names_out()`	11
PCA	`PCA(n_components=2).fit_transform(X)`	8

Task	Key code	Sets
LDA	`LinearDiscriminantAnalysis(n_components=2).fit_transform(X, y)` (needs y)	9, 13

Step 2: Q2 is one pattern

```
X, y  →  train_test_split(X, y, test_size=0.25, random_state=42)  →  model.fit  →  model.predict  →  score
```

Regression is scored with MSE and RMSE. Classification is scored with the confusion matrix plus accuracy, precision, recall and F1. "Optimise" always means `GridSearchCV` (or `RandomizedSearchCV`) with a small dictionary of parameters.

Step 3: Order for tonight

1. **Set 1** → **Set 4**: they cover most Q1 tasks plus linear, logistic and perceptron.
2. **Set 6, 12, 13**: Random Forest and AdaBoost.
3. **Set 5, 7, 15**: MLP.
4. **Set 14**: perceptron vs logistic (you'll already know both).
5. **Q1 only** of sets 8, 9, 10, 11.
6. Set 8 Q2, if you have time left.

Exam tips

- In the exam, look at your file first: `df.head()`, `df.info()`. Then change the column names in the code.
- If a question's condition doesn't match the data (for example "fill if missing > 10%" and nothing is above 10%), the code prints why it left the column alone. Say that out loud in the viva. It shows you checked.
- Precision / recall with 3 classes (IRIS) need `average="macro"`.

SET 1 EDA + Simple Linear Regression

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 3 (simple linear regression)

Datasets: Q1 `Diabetes.csv`. Q2 asks for `Rampur.csv`, which we don't have, so this uses `Kasol.csv`: same kind of river data, with the same `Wind` and `Discharge` columns. In the exam, change only the file name.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Diabetes.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(768, 9)
```

OUTPUT

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Q1.2 Display the columns with null values

```
nulls = df.isnull().sum()
print(nulls)
print("\nColumns with null values:", list(nulls[nulls > 0].index))
```

```
OUTPUT
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

Columns with null values: []
```

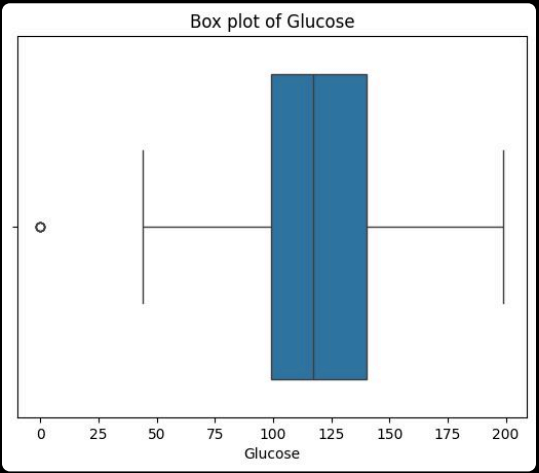
> Diabetes has no empty cells, but a **0** in Glucose, BloodPressure, SkinThickness, Insulin or BMI is impossible, so those zeros are really hidden missing values (good viva point).

Q1.3 5-number summary of a numerical column, correlated with a box plot

```
col = "Glucose"
print(df[col].describe()[["min", "25%", "50%", "75%", "max"]])     # the 5-number
   summary

sns.boxplot(x=df[col])
plt.title("Box plot of " + col)
plt.show()
```

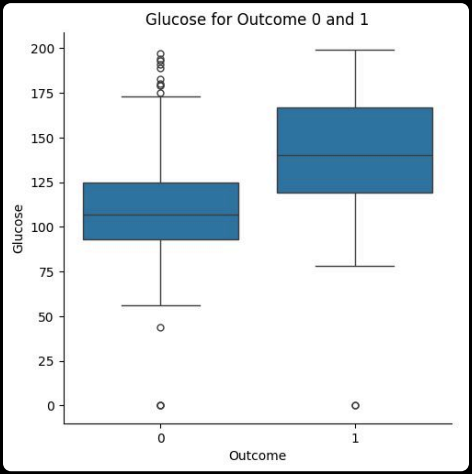
```
OUTPUT
min      0.00
25%     99.00
50%    117.00
75%    140.25
max    199.00
Name: Glucose, dtype: float64
```



Correlate with the box plot: left edge of the box = 25%, middle line = 50% (median), right edge = 75%, whiskers reach towards min and max, and dots beyond the whiskers are outliers.

Q1.4 Bivariate analysis: continuous vs categorical (categorical plot)

```
sns.catplot(x="Outcome", y="Glucose", data=df, kind="box")
plt.title("Glucose for Outcome 0 and 1")
plt.show()
```



People with diabetes (Outcome = 1) have clearly higher Glucose.

Q1.5 Drop rows with an outlier (Z-score > 3 in any column) and display the size

```
num = df.select_dtypes("number")
z = ((num - num.mean()) / num.std()).abs()      # z-score of every value
df_clean = df[(z <= 3).all(axis=1)]              # keep rows where EVERY column has
   |z| <= 3
print("Before:", df.shape)
print("After :", df_clean.shape)
```

OUTPUT

```
Before: (768, 9)
After : (688, 9)
```

Question 2

Q2.1 Read and divide into training (75%) and testing (25%)

```
kasol = pd.read_csv("data/Kasol.csv").dropna()      # drop the empty lines
X = kasol[["Wind"]]                                   # double brackets: X must be
   a table
y = kasol["Discharge"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

OUTPUT

```
Train: (5468, 1)  Test: (1823, 1)
```

Q2.2 Impact of Wind on Discharge: intercept and coefficient

```
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(X_train, y_train)
print("Intercept  :", round(model.intercept_, 4))
print("Coefficient:", model.coef_.round(4))
```

OUTPUT

```
Intercept  : 1045.3113
Coefficient: [-161.3566]
```

Q2.3 Predict discharge for the test set

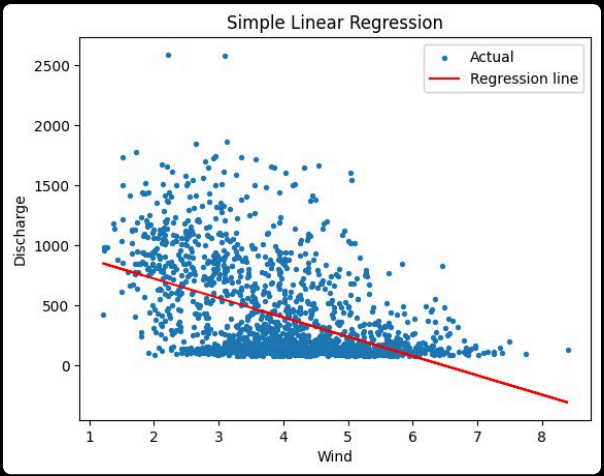
```
y_pred = model.predict(X_test)
pd.DataFrame({"Actual": y_test.values, "Predicted": y_pred}).head()
```

OUTPUT

```
   Actual   Predicted
0  128.40  456.359826
1  180.00   46.514149
2   84.20  253.050553
3   85.80  320.820311
4  778.94  336.955967
```

Q2.4 Display the regression line

```
plt.scatter(X_test, y_test, s=8, label="Actual")
plt.plot(X_test, y_pred, color="red", label="Regression line")
plt.xlabel("Wind")
plt.ylabel("Discharge")
plt.legend()
plt.title("Simple Linear Regression")
plt.show()
```



Q2.5 MSE and RMSE

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)
print("MSE :", round(mse, 2))
print("RMSE:", round(np.sqrt(mse), 2))
```

```
OUTPUT
MSE : 117031.24
RMSE: 342.1
```

> Wind alone explains discharge poorly (large RMSE), which is why Set 2 uses **many** columns.

SET 2 EDA + Multiple Linear Regression

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 3 (multiple linear regression)

Datasets: Q1 `Toyota.csv` (the paper's "toyato.csv"), Q2 `Kasol.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Toyota.csv", index_col=0, na_values=["??", "????"])   # "??"
  means missing in this file
df["Doors"] = df["Doors"].replace({"three": 3, "four": 4, "five": 5}).astype(int)
  # words -> numbers
print(df.shape)
df.head()
```

OUTPUT

```
(1436, 10)
```

OUTPUT

```
   Price   Age       KM FuelType    HP  MetColor  Automatic    CC  Doors  \
0  13500  23.0  46986.0   Diesel  90.0       1.0          0  2000      3
1  13750  23.0  72937.0   Diesel  90.0       1.0          0  2000      3
2  13950  24.0  41711.0   Diesel  90.0       NaN          0  2000      3
3  14950  26.0  48000.0   Diesel  90.0       0.0          0  2000      3
4  13750  30.0  38500.0   Diesel  90.0       0.0          0  2000      3

   Weight
0    1165
1    1165
2    1165
3    1165
4    1170
```

Q1.2 Numerical columns: min, max and mode

```
num = df.select_dtypes("number")
pd.DataFrame({"min": num.min(), "max": num.max(), "mode": num.mode().iloc[0]})
```

OUTPUT

	min	max	mode
Price	4350.0	32500.0	8950.0
Age	1.0	80.0	65.0
KM	1.0	243000.0	1.0
HP	69.0	192.0	110.0
MetColor	0.0	1.0	1.0
Automatic	0.0	1.0	0.0
CC	1300.0	2000.0	1600.0
Doors	2.0	5.0	5.0
Weight	1000.0	1615.0	1075.0

Q1.3 Number of categorical and numerical columns

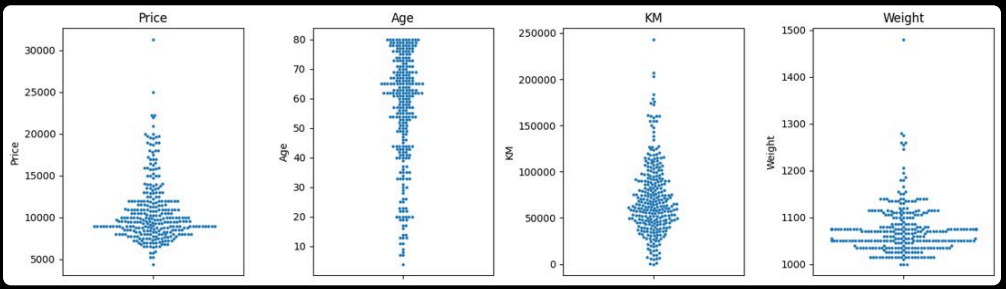
```
cat_cols = list(df.select_dtypes("object").columns) + ["MetColor", "Automatic"]   # 0/1 columns are categories too
num_cols = [c for c in df.columns if c not in cat_cols]
print("Categorical:", len(cat_cols), cat_cols)
print("Numerical  :", len(num_cols), num_cols)
```

OUTPUT

```
Categorical: 3 ['FuelType', 'MetColor', 'Automatic']
Numerical  : 7 ['Price', 'Age', 'KM', 'HP', 'CC', 'Doors', 'Weight']
```

Q1.4 Univariate analysis of continuous variables (swarm plot)

```
sample = df.sample(300, random_state=1)        # 1436 dots is too many for a swarm plot, so plot 300
fig, axes = plt.subplots(1, 4, figsize=(14, 4))
for ax, col in zip(axes, ["Price", "Age", "KM", "Weight"]):
    sns.swarmplot(y=sample[col], ax=ax, size=3)
    ax.set_title(col)
plt.tight_layout()
plt.show()
```



Q1.5 Mode imputation for categorical data if missing > 10%

```
pct = df.isnull().mean() * 100                    # % missing in every column
for col in cat_cols:
    if pct[col] > 10:
        df[col] = df[col].fillna(df[col].mode()[0])
        print(f"{col}: {pct[col]:.1f}% missing -> filled with mode")
    else:
        print(f"{col}: {pct[col]:.1f}% missing -> not more than 10%, left as it
   is")
```

```
OUTPUT
FuelType: 7.0% missing -> not more than 10%, left as it is
MetColor: 10.4% missing -> filled with mode
Automatic: 0.0% missing -> not more than 10%, left as it is
```

> Only **MetColor** (10.4%) is above 10%, so only it is filled.

Question 2

Q2.1 Read and divide into training (75%) and testing (25%)

```
kasol = pd.read_csv("data/Kasol.csv").dropna()
X = kasol.drop(columns=["Unnamed: 0", "Discharge"])     # every column except the
   date and the target
y = kasol["Discharge"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
Train: (5468, 6)  Test: (1823, 6)
```

Q2.2 Impact of multiple attributes: intercept and coefficients

```
from sklearn.linear_model import LinearRegression

model = LinearRegression().fit(X_train, y_train)
print("Intercept:", round(model.intercept_, 4))
print(pd.Series(model.coef_, index=X.columns).round(4))
```

```
OUTPUT
Intercept: 110.3711
Tmin         48.8302
Rainfall      1.6231
RH          472.0102
Solar         8.2459
Wind        -52.6075
Tmax        -21.5213
dtype: float64
```

Q2.3 Predict discharge for the test set

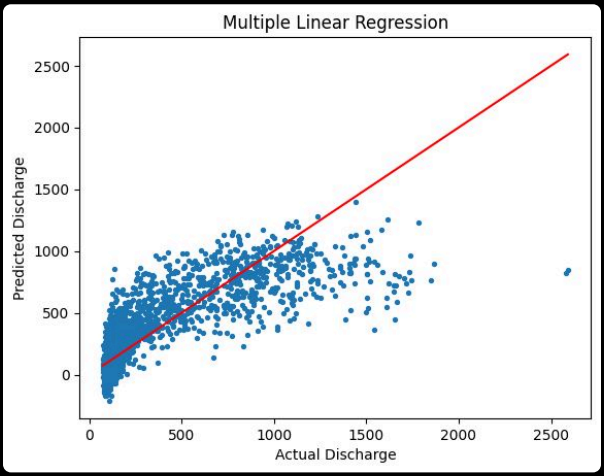
```
y_pred = model.predict(X_test)
pd.DataFrame({"Actual": y_test.values, "Predicted": y_pred}).head()
```

```
OUTPUT
   Actual   Predicted
0  128.40   49.290795
1  180.00  287.498917
2   84.20   90.012828
3   85.80  -66.524615
4  778.94  445.849585
```

Q2.4 Display the regression line

With many X columns, plot **actual vs predicted**. A perfect model puts every dot on the red line.

```
plt.scatter(y_test, y_pred, s=8)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color="red")
plt.xlabel("Actual Discharge")
plt.ylabel("Predicted Discharge")
plt.title("Multiple Linear Regression")
plt.show()
```



Q2.5 MSE and RMSE

```
from sklearn.metrics import mean_squared_error

mse = mean_squared_error(y_test, y_pred)
print("MSE :", round(mse, 2))
print("RMSE:", round(np.sqrt(mse), 2))
```

OUTPUT

```
MSE : 58601.06
RMSE: 242.08
```

SET 3 EDA + Logistic Regression

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 3 (logistic regression)

Datasets: Q1 asks for `Customers.csv`, which we don't have, so this uses `Stores.csv` (supermarket customers, with text columns and missing values). Q2 uses `Solar.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Stores.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(50, 15)
```

OUTPUT

```
  Branch       City Customer  Gender            Product line  Unit_price  \
0      A     Yangon   Member  Female       Health and beauty       74.69
1      C  Naypyitaw   Normal  Female  Electronic accessories       15.28
2      A     Yangon   Normal    Male      Home and lifestyle       46.33
3      A     Yangon   Member    Male       Health and beauty       58.22
4      A     Yangon   Normal    Male       Sports and travel       86.31

   Quantity      Tax     Total      Payment    cogs  Rating   Age  \
0         7  26.1415  548.9715      Ewallet  522.83     9.1  23.0
1         5   3.8200   80.2200         Cash   76.40     9.6  23.0
2         7  16.2155  340.5255  Credit card  324.31     7.4  24.0
3         8  23.2880  489.0480      Ewallet  465.76     8.4  26.0
4        60  30.2085  634.3785      Ewallet  604.17     5.3  30.0

   Quarterly_Tax  Price
0          210.0   7469
... (4 more lines)
```

Q1.2 Features with high positive, high negative and no correlation

```
corr = df.select_dtypes("number").corr()
print(corr.round(2), "\n")
found = {"high positive": [], "high negative": [], "no correlation": []}
for a in corr.columns:
    for b in corr.columns:
        if a < b:                                   # each pair once
            r = corr.loc[a, b]
            if r > 0.5:
                found["high positive"].append(f"{a} & {b} ({r:.2f})")
            elif r < -0.5:
                found["high negative"].append(f"{a} & {b} ({r:.2f})")
            elif abs(r) < 0.2:
                found["no correlation"].append(f"{a} & {b} ({r:.2f})")
for label in ["high positive", "high negative", "no correlation"]:
    print(label.upper(), ":", found[label] if found[label] else "none")
```

OUTPUT

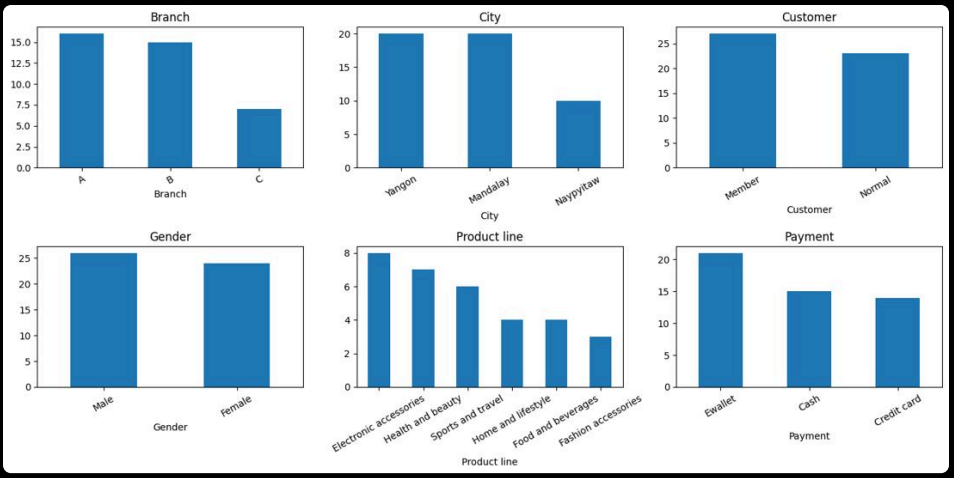
```
              Unit_price  Quantity   Tax  Total  cogs  Rating   Age  \
Unit_price          1.00      0.05  0.61   0.61  0.61   -0.12  0.20
Quantity            0.05      1.00  0.23   0.23  0.23   -0.30  0.03
Tax                 0.61      0.23  1.00   1.00  1.00   -0.10  0.29
Total               0.61      0.23  1.00   1.00  1.00   -0.10  0.29
cogs                0.61      0.23  1.00   1.00  1.00   -0.10  0.31
Rating             -0.12     -0.30 -0.10  -0.10 -0.10    1.00 -0.16
Age                 0.20      0.03  0.29   0.29  0.31   -0.16  1.00
Quarterly_Tax      -0.14      0.03  0.15   0.15  0.15    0.38 -0.25
Price               1.00      0.05  0.61   0.61  0.61   -0.12  0.20

              Quarterly_Tax  Price
Unit_price            -0.14   1.00
Quantity               0.03   0.05
Tax                    0.15   0.61
Total                  0.15   0.61
... (9 more lines)
```

Rule used: r > 0.5 = high positive, r < −0.5 = high negative, |r| < 0.2 = no correlation (close to 0).

Q1.3 Univariate analysis of categorical variables (bar plot)

```
cat_cols = df.select_dtypes("object").columns
fig, axes = plt.subplots(2, 3, figsize=(14, 7))
for ax, col in zip(axes.ravel(), cat_cols):
    df[col].value_counts().plot(kind="bar", ax=ax, title=col, rot=30)
plt.tight_layout()
plt.show()
```



Q1.4 Display features with missing values > 10% and drop those columns

```
pct = df.isnull().mean() * 100
print(pct[pct > 0].round(1))
drop_cols = pct[pct > 10].index
print("\nDropping:", list(drop_cols))
df = df.drop(columns=drop_cols)
```

```
OUTPUT
Branch           24.0
Product line     36.0
cogs              6.0
Rating            4.0
Age               6.0
Quarterly_Tax     4.0
dtype: float64

Dropping: ['Branch', 'Product line']
```

Q1.5 Replace missing values with the mean for numerical columns with < 10% missing

```
pct = df.isnull().mean() * 100                    # % missing in every column
for col in df.select_dtypes("number").columns:
    if 0 < pct[col] < 10:
        df[col] = df[col].fillna(df[col].mean())
        print(f"{col}: {pct[col]:.1f}% missing -> filled with mean")
    else:
        print(f"{col}: {pct[col]:.1f}% missing -> not less than 10%, left as it
  is")
```

OUTPUT

```
Unit_price: 0.0% missing -> not less than 10%, left as it is
Quantity: 0.0% missing -> not less than 10%, left as it is
Tax: 0.0% missing -> not less than 10%, left as it is
Total: 0.0% missing -> not less than 10%, left as it is
cogs: 6.0% missing -> filled with mean
Rating: 4.0% missing -> filled with mean
Age: 6.0% missing -> filled with mean
Quarterly_Tax: 4.0% missing -> filled with mean
Price: 0.0% missing -> not less than 10%, left as it is
```

```
print("Missing values left:", df.isnull().sum().sum())
```

OUTPUT

```
Missing values left: 0
```

Question 2

Q2.1 Divide into training (75%) and testing (25%)

`Solar Radiation` is already 0 / 1, so it's the 2 categories.

```
from sklearn.preprocessing import StandardScaler

solar = pd.read_csv("data/Solar.csv")
X = StandardScaler().fit_transform(solar.drop(columns="Solar Radiation"))
y = solar["Solar Radiation"]
print(y.value_counts().to_dict())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

OUTPUT

```
{0: 114, 1: 114}
Train: (171, 6)  Test: (57, 6)
```

Q2.2 Design the classifier and evaluate on the test set

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression().fit(X_train, y_train)
```

Q2.3 Confusion table and Accuracy, Precision, Recall, F-measure

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

y_pred = model.predict(X_test)
print("Confusion table:")
print(confusion_matrix(y_test, y_pred))
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred, average="macro"), 4))
print("Recall   :", round(recall_score(y_test, y_pred, average="macro"), 4))
print("F-measure:", round(f1_score(y_test, y_pred, average="macro"), 4))
```

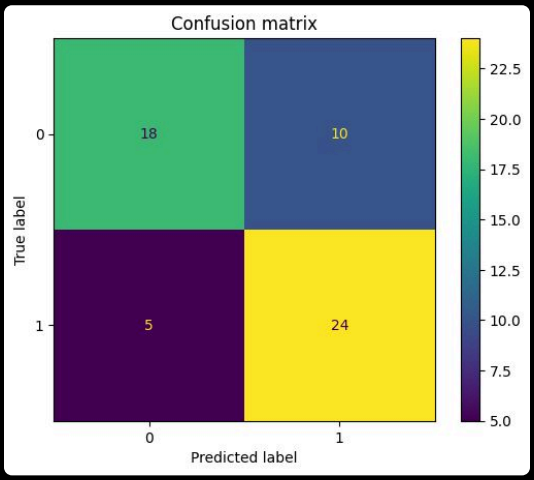
OUTPUT

```
Confusion table:
[[18 10]
 [ 5 24]]
Accuracy : 0.7368
Precision: 0.7442
Recall   : 0.7352
F-measure: 0.7339
```

Q2.4 Plot the results

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion matrix")
plt.show()
```



SET 4 EDA + Single Layer Perceptron

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 4 (perceptron)

Datasets: Q1 `Stores.csv`, Q2 `Diabetes.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Stores.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(50, 15)
```

OUTPUT

```
  Branch       City Customer  Gender            Product line  Unit_price  \
0      A     Yangon   Member  Female       Health and beauty       74.69
1      C  Naypyitaw   Normal  Female  Electronic accessories       15.28
2      A     Yangon   Normal    Male      Home and lifestyle       46.33
3      A     Yangon   Member    Male       Health and beauty       58.22
4      A     Yangon   Normal    Male       Sports and travel       86.31

   Quantity      Tax     Total      Payment    cogs  Rating   Age  \
0         7  26.1415  548.9715      Ewallet  522.83     9.1  23.0
1         5   3.8200   80.2200         Cash   76.40     9.6  23.0
2         7  16.2155  340.5255  Credit card  324.31     7.4  24.0
3         8  23.2880  489.0480      Ewallet  465.76     8.4  26.0
4        60  30.2085  634.3785      Ewallet  604.17     5.3  30.0

   Quarterly_Tax  Price
0          210.0   7469
... (4 more lines)
```

Q1.2 Identify the numerical columns: min, max and mean

```
num = df.select_dtypes("number")
print("Numerical columns:", list(num.columns))
pd.DataFrame({"min": num.min(), "max": num.max(), "mean": num.mean().round(2)})
```

```
OUTPUT
Numerical columns: ['Unit_price', 'Quantity', 'Tax', 'Total', 'cogs', 'Rating', 'Age',
'Quarterly_Tax', 'Price']
```

```
OUTPUT
                    min      max     mean
Unit_price       14.480    99.42    60.24
Quantity          1.000    60.00     9.48
Tax               1.537    39.48    16.51
Total            32.277   829.08   346.67
cogs             30.740   789.60   331.29
Rating            4.100     9.90     6.80
Age              22.000    32.00    26.26
Quarterly_Tax    19.000   234.00   124.00
Price          1448.000  9942.00  6024.16
```

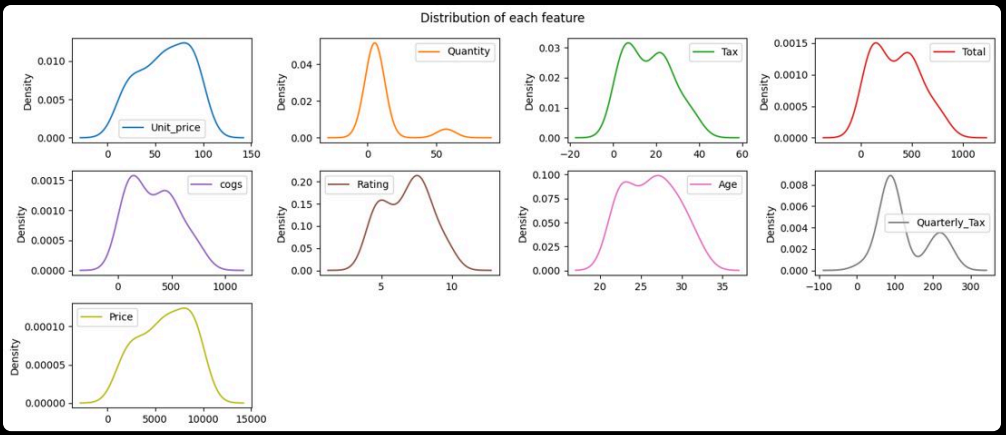
Q1.3 Skewness with distribution plots: right, left and no skew

```
num = df.select_dtypes("number")
skew = num.skew()
print(skew.round(2))
print("\nRight skew (> 0.5):", list(skew[skew > 0.5].index))
print("Left skew (< -0.5):", list(skew[skew < -0.5].index))
print("No skew  (-0.5 to 0.5):", list(skew[skew.abs() <= 0.5].index))

num.plot(kind="kde", subplots=True, layout=(-1, 4), figsize=(14, 6), sharex=False,
   title="Distribution of each feature")
plt.tight_layout()
plt.show()
```

```
OUTPUT
Unit_price      -0.26
Quantity         3.02
Tax              0.35
Total            0.35
cogs             0.36
Rating          -0.01
Age              0.15
Quarterly_Tax    0.76
Price           -0.26
dtype: float64

Right skew (> 0.5): ['Quantity', 'Quarterly_Tax']
Left skew (< -0.5): []
No skew  (-0.5 to 0.5): ['Unit_price', 'Tax', 'Total', 'cogs', 'Rating', 'Age', 'Price']
```



Right skew = long tail on the right (mean > median). Left skew = long tail on the left (mean < median).

Q1.4 Mean imputation for numerical data if missing > 10%

```
pct = df.isnull().mean() * 100                    # % missing in every column
for col in df.select_dtypes("number").columns:
    if pct[col] > 10:
        df[col] = df[col].fillna(df[col].mean())
        print(f"{col}: {pct[col]:.1f}% missing -> filled with mean")
    else:
        print(f"{col}: {pct[col]:.1f}% missing -> not more than 10%, left as it
  is")
```

OUTPUT

```
Unit_price: 0.0% missing -> not more than 10%, left as it is
Quantity: 0.0% missing -> not more than 10%, left as it is
Tax: 0.0% missing -> not more than 10%, left as it is
Total: 0.0% missing -> not more than 10%, left as it is
cogs: 6.0% missing -> not more than 10%, left as it is
Rating: 4.0% missing -> not more than 10%, left as it is
Age: 6.0% missing -> not more than 10%, left as it is
Quarterly_Tax: 4.0% missing -> not more than 10%, left as it is
Price: 0.0% missing -> not more than 10%, left as it is
```

No numerical column here is above 10%, so the code correctly leaves them. With your exam file, the same code fills the ones above 10%.

Q1.5 Scaling with StandardScaler

```
from sklearn.preprocessing import StandardScaler

num = df.select_dtypes("number")
scaled = pd.DataFrame(StandardScaler().fit_transform(num), columns=num.columns)
print(scaled.describe().loc[["mean", "std"]].round(2))      # every column now has
   mean 0, std 1
scaled.head()
```

OUTPUT

```
      Unit_price  Quantity   Tax  Total  cogs  Rating   Age  Quarterly_Tax  \
mean        0.00     -0.00 -0.00   0.00 -0.00   -0.00  0.00           0.00
std         1.01      1.01  1.01   1.01  1.01    1.01  1.01           1.01

      Price
mean   0.00
std    1.01
```

OUTPUT

```
   Unit_price  Quantity       Tax     Total      cogs    Rating       Age  \
0    0.555602 -0.174041  0.906524  0.906524  0.887110  1.411434 -1.046327
1   -1.728964 -0.314396 -1.193984 -1.193984 -1.180517  1.718824 -1.046327
2   -0.534960 -0.174041 -0.027537 -0.027537 -0.032330  0.366307 -0.724906
3   -0.077739 -0.103863  0.638002  0.638002  0.622792  0.981087 -0.082065
4    1.002441  3.545376  1.289239  1.289239  1.263833 -0.924733  1.203618

   Quarterly_Tax     Price
0       1.365804  0.555602
1       1.365804 -1.728964
2       1.365804 -0.534960
3       1.365804 -0.077739
4       1.365804  1.002441
```

Question 2

Q2.1 Read and divide into training (70%) and testing (30%)

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/Diabetes.csv")
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
   # a 0 here is impossible = missing
df[zero_is_missing] = df[zero_is_missing].replace(0, np.nan)
df = df.fillna(df.median())                          # fill with the median
X = StandardScaler().fit_transform(df.drop(columns="Outcome"))
y = df["Outcome"]
print(X.shape, y.value_counts().to_dict())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
(768, 8) {0: 500, 1: 268}
Train: (537, 8)  Test: (231, 8)
```

Q2.2 Build the model

```
from sklearn.linear_model import Perceptron

model = Perceptron(eta0=0.1, random_state=42).fit(X_train, y_train)
print("Weights:", model.coef_.round(3))
print("Bias   :", model.intercept_.round(3))
```

```
OUTPUT
Weights: [[ 0.114  0.341 -0.245  0.081 -0.061  0.334  0.18  -0.006]]
Bias   : [-0.1]
```

Q2.3 Evaluate on the test set: confusion table, Accuracy, Precision, Recall, F-measure

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

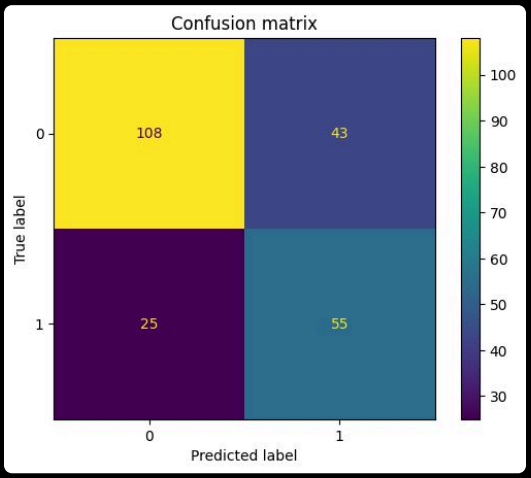
y_pred = model.predict(X_test)
print("Confusion table:")
print(confusion_matrix(y_test, y_pred))
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred, average="macro"), 4))
print("Recall   :", round(recall_score(y_test, y_pred, average="macro"), 4))
print("F-measure:", round(f1_score(y_test, y_pred, average="macro"), 4))
```

```
OUTPUT
Confusion table:
[[108  43]
 [ 25  55]]
Accuracy : 0.7056
Precision: 0.6866
Recall   : 0.7014
F-measure: 0.6893
```

Q2.4 Plot the results

```
from sklearn.metrics import ConfusionMatrixDisplay

ConfusionMatrixDisplay.from_predictions(y_test, y_pred)
plt.title("Confusion matrix")
plt.show()
```



SET 5 EDA + MLP Regression with Randomized / Grid Search

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 5 (MLP regression)

Datasets: the paper asks for `Solar-Regression.csv`, which we don't have, so this uses `Kasol.csv` for both questions. Its `Solar` column is a continuous solar radiation value, which is what an MLP regressor needs.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Kasol.csv").dropna(how="all")    # the file has empty ",,,,"
  lines: drop rows that are ALL empty
df = df.rename(columns={"Unnamed: 0": "Date"})
print(df.shape)
df.head()
```

OUTPUT

```
(7305, 8)
```

OUTPUT

	Date	Tmin	Rainfall	RH	Solar	Wind	Tmax	Discharge
0	1/1/1991	3.3	0.0	0.47	7.54	2.59	11.1	133.5
1	1/2/1991	4.4	0.0	0.24	13.55	3.05	8.9	122.0
2	1/3/1991	4.4	0.0	0.33	13.37	3.34	15.6	114.8
3	1/4/1991	4.4	0.0	0.30	9.45	3.07	16.7	114.5
4	1/5/1991	5.6	0.0	0.21	13.60	3.89	16.7	113.9

Q1.2 Features with high positive correlation and no correlation

```
corr = df.select_dtypes("number").corr()
print(corr.round(2), "\n")
found = {"high positive": [], "high negative": [], "no correlation": []}
for a in corr.columns:
    for b in corr.columns:
        if a < b:                                   # each pair once
            r = corr.loc[a, b]
            if r > 0.5:
                found["high positive"].append(f"{a} & {b} ({r:.2f})")
            elif r < -0.5:
                found["high negative"].append(f"{a} & {b} ({r:.2f})")
            elif abs(r) < 0.2:
                found["no correlation"].append(f"{a} & {b} ({r:.2f})")
for label in ["high positive", "no correlation"]:
    print(label.upper(), ":", found[label] if found[label] else "none")
```

OUTPUT

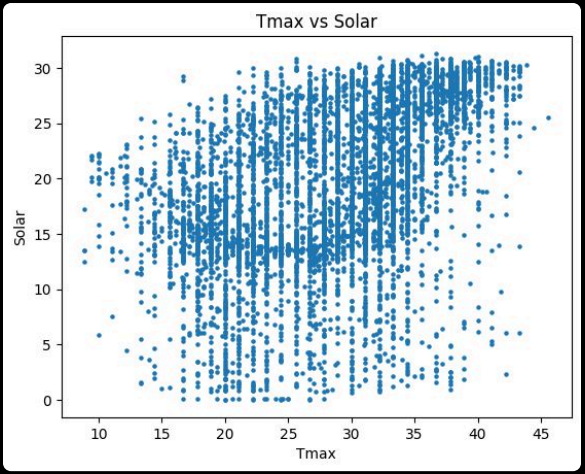
```
            Tmin  Rainfall    RH  Solar  Wind  Tmax  Discharge
Tmin        1.00      0.18  0.27   0.47 -0.32  0.87       0.70
Rainfall    0.18      1.00  0.14   0.03 -0.14  0.10       0.22
RH          0.27      0.14  1.00  -0.26 -0.62  0.12       0.49
Solar       0.47      0.03 -0.26   1.00  0.17  0.48       0.27
Wind       -0.32     -0.14 -0.62   0.17  1.00 -0.18      -0.49
Tmax        0.87      0.10  0.12   0.48 -0.18  1.00       0.49
Discharge   0.70      0.22  0.49   0.27 -0.49  0.49       1.00

HIGH POSITIVE : ['Tmax & Tmin (0.87)', 'Discharge & Tmin (0.70)']
NO CORRELATION : ['Rainfall & Tmin (0.18)', 'Rainfall & Solar (0.03)', 'Rainfall & Wind
(-0.14)', 'Rainfall & Tmax (0.10)', 'RH & Rainfall (0.14)', 'RH & Tmax (0.12)', 'Solar &
Wind (0.17)', 'Tmax & Wind (-0.18)']
```

> Rule used: r > 0.5 = high positive, r < −0.5 = high negative, |r| < 0.2 = no correlation (close to 0).

Q1.3 Scatter plot of two continuous variables

```
plt.scatter(df["Tmax"], df["Solar"], s=5)
plt.xlabel("Tmax")
plt.ylabel("Solar")
plt.title("Tmax vs Solar")
plt.show()
```



Q1.4 Median imputation for numerical data if missing > 10%

```
pct = df.isnull().mean() * 100                    # % missing in every column
for col in df.select_dtypes("number").columns:
    if pct[col] > 10:
        df[col] = df[col].fillna(df[col].median())
        print(f"{col}: {pct[col]:.1f}% missing -> filled with median")
    else:
        print(f"{col}: {pct[col]:.1f}% missing -> not more than 10%, left as it
  is")
```

OUTPUT

```
Tmin: 0.0% missing -> not more than 10%, left as it is
Rainfall: 0.2% missing -> not more than 10%, left as it is
RH: 0.0% missing -> not more than 10%, left as it is
Solar: 0.0% missing -> not more than 10%, left as it is
Wind: 0.0% missing -> not more than 10%, left as it is
Tmax: 0.0% missing -> not more than 10%, left as it is
Discharge: 0.0% missing -> not more than 10%, left as it is
```

Q1.5 Remove features with missing values

```
print("Before:", list(df.columns))
df = df.dropna(axis=1)          # axis=1 = drop COLUMNS that still have a missing
  value
print("After :", list(df.columns))
```

OUTPUT

```
Before: ['Date', 'Tmin', 'Rainfall', 'RH', 'Solar', 'Wind', 'Tmax', 'Discharge']
After : ['Date', 'Tmin', 'RH', 'Solar', 'Wind', 'Tmax', 'Discharge']
```

Question 2

Q2.1 Read and divide into training (75%) and testing (25%), with preprocessing (scaling)

```
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = pd.read_csv("data/Kasol.csv").dropna()                # drop the empty
   lines (and 14 rows with missing Rainfall)
X = StandardScaler().fit_transform(data.drop(columns=["Unnamed: 0", "Solar"]))   #
   drop the date and the target
y = data["Solar"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
Train: (5468, 6)  Test: (1823, 6)
```

Q2.2 Build the model and evaluate on the test set (RMSE)

```
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error

model = MLPRegressor(hidden_layer_sizes=(32,), max_iter=300, early_stopping=True,
   random_state=42).fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("MSE :", round(mse, 4))
print("RMSE:", round(np.sqrt(mse), 4))
```

```
OUTPUT
MSE : 18.9464
RMSE: 4.3527
```

Q2.3 Improve with Randomized Search CV

```
from sklearn.model_selection import RandomizedSearchCV

params = {"hidden_layer_sizes": [(32,), (64,), (64, 32)], "alpha": [0.0001, 0.001,
   0.01], "learning_rate_init": [0.001, 0.01]}
rand = RandomizedSearchCV(MLPRegressor(max_iter=300, early_stopping=True,
   random_state=42), params,
                          n_iter=5, cv=3, random_state=42).fit(X_train, y_train)
print("Optimal hyperparameters (Randomized):", rand.best_params_)
rmse_rand = np.sqrt(mean_squared_error(y_test, rand.predict(X_test)))
print("RMSE after Randomized Search:", round(rmse_rand, 4))
```

```
OUTPUT
Optimal hyperparameters (Randomized): {'learning_rate_init': 0.01, 'hidden_layer_sizes':
(64, 32), 'alpha': 0.0001}
RMSE after Randomized Search: 4.3519
```

Q2.4 Improve with Grid Search CV

```
from sklearn.model_selection import GridSearchCV

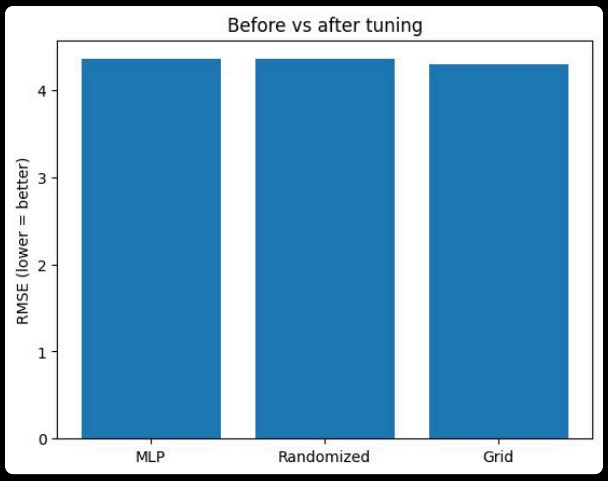
grid = GridSearchCV(MLPRegressor(max_iter=300, early_stopping=True,
   random_state=42),
                    {"hidden_layer_sizes": [(32,), (64, 32)], "alpha": [0.0001,
   0.01]}, cv=3).fit(X_train, y_train)
print("Optimal hyperparameters (Grid):", grid.best_params_)
rmse_grid = np.sqrt(mean_squared_error(y_test, grid.predict(X_test)))
print("RMSE after Grid Search:", round(rmse_grid, 4))
```

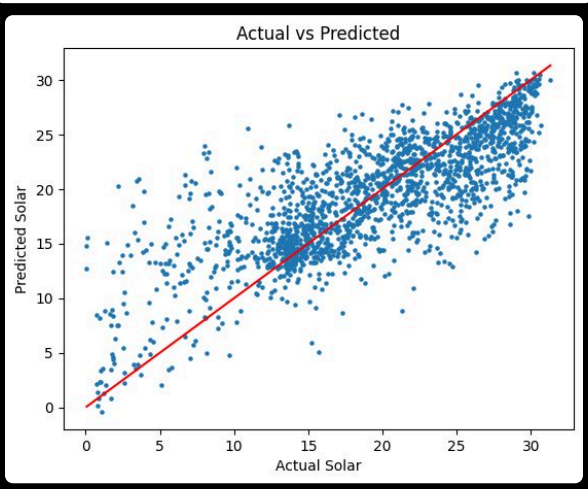
```
OUTPUT
Optimal hyperparameters (Grid): {'alpha': 0.0001, 'hidden_layer_sizes': (64, 32)}
RMSE after Grid Search: 4.2939
```

Q2.5 Plot the results

```
rmse = {"MLP": np.sqrt(mse), "Randomized": rmse_rand, "Grid": rmse_grid}
plt.bar(rmse.keys(), rmse.values())
plt.ylabel("RMSE (lower = better)")
plt.title("Before vs after tuning")
plt.show()

plt.scatter(y_test, grid.predict(X_test), s=5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color="red")
plt.xlabel("Actual Solar")
plt.ylabel("Predicted Solar")
plt.title("Actual vs Predicted")
plt.show()
```


Before vs after tuning
RMSE (lower = better)
0
1
2
3
4
MLP
Randomized
Grid


Actual vs Predicted
Predicted Solar
Actual Solar
0
5
10
15
20
25
30

SET 6 EDA + Random Forest (optimised)

LEARN Q2

Q1 → Exp 1A / 1B / 2 · Q2 → Exp 10 (Random Forest)

Datasets: Q1 asks for `Precipitation.csv`, which we don't have, so this uses `Toyota.csv`: it has numerical **and** text columns, which this question needs. Q2 uses `Diabetes.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Toyota.csv", index_col=0, na_values=["??", "????"])   # "??"
  means missing in this file
df["Doors"] = df["Doors"].replace({"three": 3, "four": 4, "five": 5}).astype(int)
  # words -> numbers
print(df.shape)
df.head()
```

OUTPUT

```
(1436, 10)
```

OUTPUT

	Price	Age	KM	FuelType	HP	MetColor	Automatic	CC	Doors	\
0	13500	23.0	46986.0	Diesel	90.0	1.0	0	2000	3	
1	13750	23.0	72937.0	Diesel	90.0	1.0	0	2000	3	
2	13950	24.0	41711.0	Diesel	90.0	NaN	0	2000	3	
3	14950	26.0	48000.0	Diesel	90.0	0.0	0	2000	3	
4	13750	30.0	38500.0	Diesel	90.0	0.0	0	2000	3	

	Weight
0	1165
1	1165
2	1165
3	1165
4	1170

Q1.2 Display the outlier values of numerical columns using Z-score

```
num = df.select_dtypes("number")
z = ((num - num.mean()) / num.std()).abs()      # z-score: how many std-devs each
   value is from its column mean
for col in num.columns:
    outliers = num[col][z[col] > 3]
    print(f"{col}: {len(outliers)} outliers ->", outliers.values[:10])
```

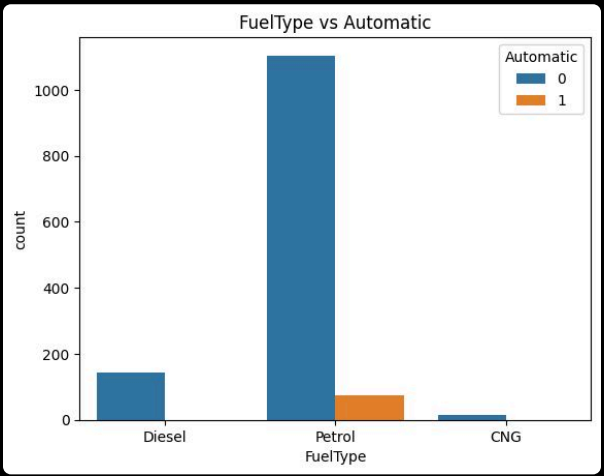
OUTPUT

```
Price: 26 outliers -> [22500 22000 22750 21950 21950 22250 21950 22250 32500 31000]
Age: 0 outliers -> []
KM: 17 outliers -> [243000. 216000. 198167. 205000. 204250. 194545. 190900. 183500.
232940.
 218118.]
HP: 10 outliers -> [192. 192. 192. 192. 192. 192. 192. 192. 192. 192.]
MetColor: 0 outliers -> []
Automatic: 80 outliers -> [1 1 1 1 1 1 1 1 1 1]
CC: 0 outliers -> []
Doors: 0 outliers -> []
Weight: 30 outliers -> [1245 1245 1255 1255 1270 1255 1255 1255 1275 1245]
```

> |z| > 3 means the value is more than 3 standard deviations from the mean, so it's an outlier.

Q1.3 Bivariate analysis of two categorical variables (count plot)

```
sns.countplot(x="FuelType", hue="Automatic", data=df)
plt.title("FuelType vs Automatic")
plt.show()
```



Q1.4 Identify columns with variance < 10 and drop them

```
var = df.select_dtypes("number").var()
print(var.round(2))
low = var[var < 10].index
print("\nDropping (variance < 10):", list(low))
df = df.drop(columns=low)
```

OUTPUT

```
Price        1.315487e+07
Age          3.455800e+02
KM           1.393755e+09
HP           2.181000e+02
MetColor     2.200000e-01
Automatic    5.000000e-02
CC           3.503726e+04
Doors        9.100000e-01
Weight       2.771090e+03
dtype: float64

Dropping (variance < 10): ['MetColor', 'Automatic', 'Doors']
```

Q1.5 Label encoding of a categorical feature

```
df["FuelType"] = df["FuelType"].fillna(df["FuelType"].mode()[0])   # fill blanks
   first, or "nan" becomes a category
from sklearn.preprocessing import LabelEncoder

print("Before:", df["FuelType"].unique()[:6])
df["FuelType"] = LabelEncoder().fit_transform(df["FuelType"].astype(str))
print("After :", df["FuelType"].unique()[:6])
```

OUTPUT

```
Before: ['Diesel' 'Petrol' 'CNG']
After : [1 2 0]
```

Question 2

Q2.1 Read, preprocess and divide into training (75%) and testing (25%)

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/Diabetes.csv")
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
   # a 0 here is impossible = missing
df[zero_is_missing] = df[zero_is_missing].replace(0, np.nan)
df = df.fillna(df.median())                          # fill with the median
X = StandardScaler().fit_transform(df.drop(columns="Outcome"))
y = df["Outcome"]
print(X.shape, y.value_counts().to_dict())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
(768, 8) {0: 500, 1: 268}
Train: (576, 8)  Test: (192, 8)
```

Q2.2 Build the model and evaluate: confusion table and Accuracy

```
from sklearn.ensemble import RandomForestClassifier

model = RandomForestClassifier(random_state=42).fit(X_train, y_train)
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

y_pred = model.predict(X_test)
print("Confusion table:")
print(confusion_matrix(y_test, y_pred))
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred, average="macro"), 4))
print("Recall   :", round(recall_score(y_test, y_pred, average="macro"), 4))
print("F-measure:", round(f1_score(y_test, y_pred, average="macro"), 4))
```

```
OUTPUT
Confusion table:
[[97 26]
 [22 47]]
Accuracy : 0.75
Precision: 0.7295
Recall   : 0.7349
F-measure: 0.7318
```

Q2.3 Optimise (GridSearchCV), compare and plot

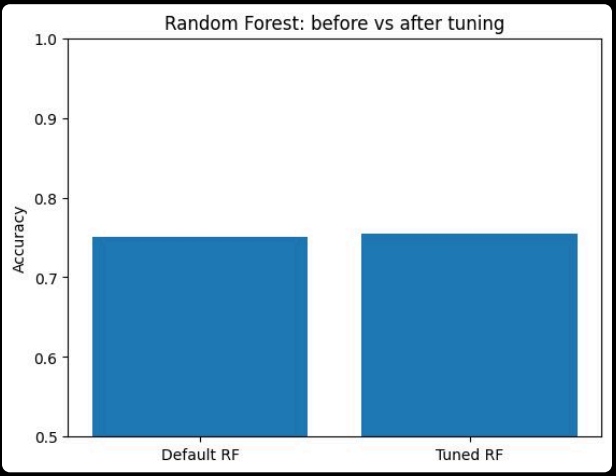
```
from sklearn.model_selection import GridSearchCV

grid = {"n_estimators": [50, 100, 200], "max_depth": [None, 4, 8], "criterion":
   ["gini", "entropy"]}
search = GridSearchCV(RandomForestClassifier(random_state=42), grid, cv=5,
   n_jobs=-1).fit(X_train, y_train)
print("Best parameters:", search.best_params_)

scores = {"Default RF": accuracy_score(y_test, y_pred), "Tuned RF":
   accuracy_score(y_test, search.predict(X_test))}
print(scores)
plt.bar(scores.keys(), scores.values())
plt.ylim(0.5, 1)
plt.ylabel("Accuracy")
plt.title("Random Forest: before vs after tuning")
plt.show()
```

OUTPUT

```
Best parameters: {'criterion': 'entropy', 'max_depth': None, 'n_estimators': 100}
{'Default RF': 0.75, 'Tuned RF': 0.7552083333333334}
```



SET 7 EDA + MLP Classifier (optimised)

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 5 (MLP)

Datasets: Q1 `IRIS.csv`, Q2 `Diabetes.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/IRIS.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(150, 5)
```

OUTPUT

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

Q1.2 Features with high negative correlation and no correlation

```
corr = df.select_dtypes("number").corr()
print(corr.round(2), "\n")
found = {"high positive": [], "high negative": [], "no correlation": []}
for a in corr.columns:
    for b in corr.columns:
        if a < b:                                   # each pair once
            r = corr.loc[a, b]
            if r > 0.5:
                found["high positive"].append(f"{a} & {b} ({r:.2f})")
            elif r < -0.5:
                found["high negative"].append(f"{a} & {b} ({r:.2f})")
            elif abs(r) < 0.2:
                found["no correlation"].append(f"{a} & {b} ({r:.2f})")
for label in ["high negative", "no correlation"]:
    print(label.upper(), ":", found[label] if found[label] else "none")
```

OUTPUT

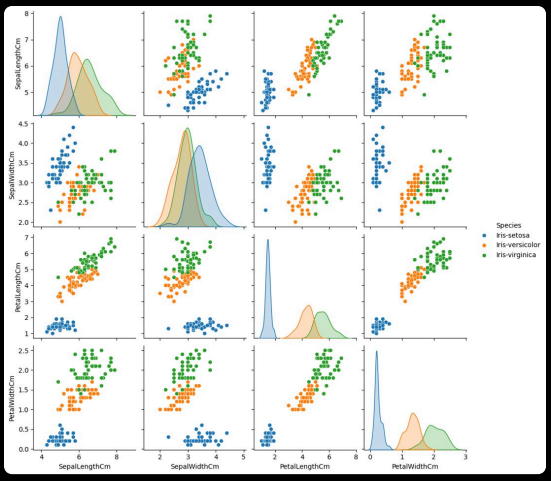
```
              SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm
SepalLengthCm          1.00         -0.12           0.87          0.82
SepalWidthCm          -0.12          1.00          -0.43         -0.37
PetalLengthCm          0.87         -0.43           1.00          0.96
PetalWidthCm           0.82         -0.37           0.96          1.00

HIGH NEGATIVE : none
NO CORRELATION : ['SepalLengthCm & SepalWidthCm (-0.12)']
```

Rule used: r > 0.5 = high positive, r < −0.5 = high negative, |r| < 0.2 = no correlation (close to 0). IRIS has no pair below −0.5, so high negative is **none**. The most negative pair is SepalWidth & PetalLength (−0.43). Say this in the viva.

Q1.3 Multivariate analysis (pair plot)

```
sns.pairplot(df, hue="Species")
plt.show()
```



Q1.4 One-hot encoding of a categorical feature

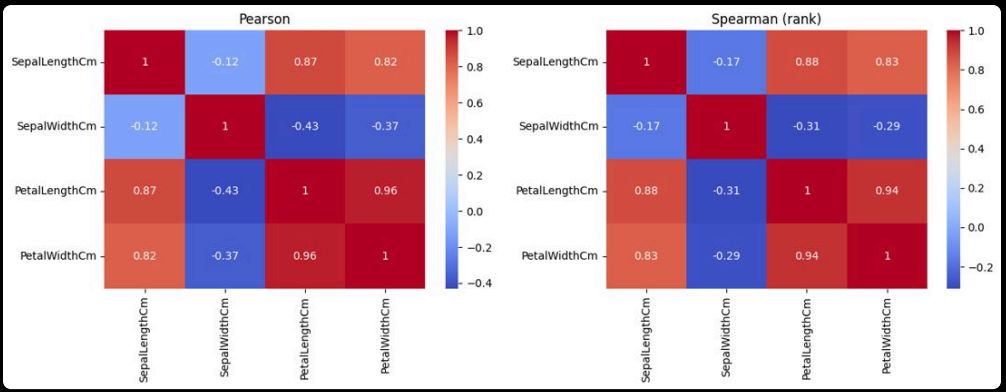
```
encoded = pd.get_dummies(df, columns=["Species"], dtype=int)    # one 0/1 column
  per species
encoded.head()
```

```
OUTPUT
   SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  \
0            5.1           3.5            1.4           0.2
1            4.9           3.0            1.4           0.2
2            4.7           3.2            1.3           0.2
3            4.6           3.1            1.5           0.2
4            5.0           3.6            1.4           0.2

   Species_Iris-setosa  Species_Iris-versicolor  Species_Iris-virginica
0                    1                        0                       0
1                    1                        0                       0
2                    1                        0                       0
3                    1                        0                       0
4                    1                        0                       0
```

Q1.5 Pearson and Spearman correlation matrix with heatmap

```
num = df.select_dtypes("number")
fig, axes = plt.subplots(1, 2, figsize=(13, 5))
sns.heatmap(num.corr(method="pearson"), annot=True, cmap="coolwarm", ax=axes[0])
axes[0].set_title("Pearson")
sns.heatmap(num.corr(method="spearman"), annot=True, cmap="coolwarm", ax=axes[1])
axes[1].set_title("Spearman (rank)")
plt.tight_layout()
plt.show()
```



Question 2

Q2.1 Read, preprocess and divide into training (75%) and testing (25%)

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/Diabetes.csv")
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
   # a 0 here is impossible = missing
df[zero_is_missing] = df[zero_is_missing].replace(0, np.nan)
df = df.fillna(df.median())                              # fill with the median
X = StandardScaler().fit_transform(df.drop(columns="Outcome"))
y = df["Outcome"]
print(X.shape, y.value_counts().to_dict())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
(768, 8) {0: 500, 1: 268}
Train: (576, 8)  Test: (192, 8)
```

Q2.2 Build the MLP and evaluate: confusion table and Accuracy

```
from sklearn.neural_network import MLPClassifier

model = MLPClassifier(hidden_layer_sizes=(32, 16), max_iter=1000,
   random_state=42).fit(X_train, y_train)
```

```
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

y_pred = model.predict(X_test)
print("Confusion table:")
print(confusion_matrix(y_test, y_pred))
print("Accuracy :", round(accuracy_score(y_test, y_pred), 4))
print("Precision:", round(precision_score(y_test, y_pred, average="macro"), 4))
print("Recall   :", round(recall_score(y_test, y_pred, average="macro"), 4))
print("F-measure:", round(f1_score(y_test, y_pred, average="macro"), 4))
```

OUTPUT

```
Confusion table:
[[94 29]
 [30 39]]
Accuracy : 0.6927
Precision: 0.6658
Recall   : 0.6647
F-measure: 0.6652
```

Q2.3 Optimise (GridSearchCV), compare and plot

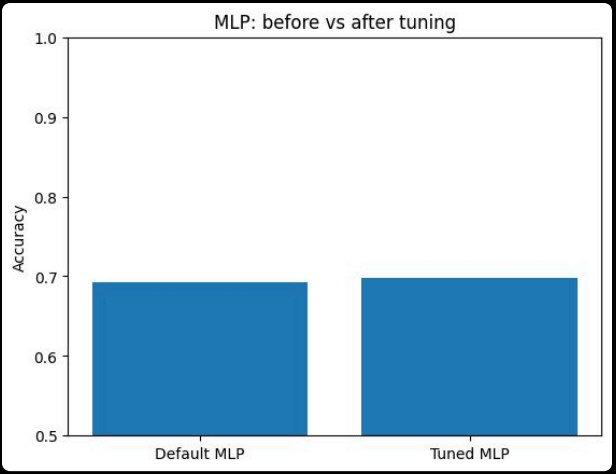
```
from sklearn.model_selection import GridSearchCV

grid = {"hidden_layer_sizes": [(16,), (32, 16)], "alpha": [0.0001, 0.01, 0.1],
   "learning_rate_init": [0.001, 0.01]}
search = GridSearchCV(MLPClassifier(max_iter=1000, random_state=42), grid, cv=5,
   n_jobs=-1).fit(X_train, y_train)
print("Best parameters:", search.best_params_)

scores = {"Default MLP": accuracy_score(y_test, y_pred), "Tuned MLP":
   accuracy_score(y_test, search.predict(X_test))}
print(scores)
plt.bar(scores.keys(), scores.values())
plt.ylim(0.5, 1)
plt.ylabel("Accuracy")
plt.title("MLP: before vs after tuning")
plt.show()
```

OUTPUT

```
Best parameters: {'alpha': 0.01, 'hidden_layer_sizes': (16,), 'learning_rate_init':
0.01}
{'Default MLP': 0.6927083333333334, 'Tuned MLP': 0.6979166666666666}
```


MLP: before vs after tuning
Accuracy
1.0
0.9
0.8
0.7
0.6
0.5
Default MLP
Tuned MLP

SET 8 EDA + Agglomerative Clustering

Q2 IF TIME

Q1 → Exp 1A / 1B / 2 · Q2 → Exp 6 (hierarchical clustering)

Datasets: Q1 `Diabetes.csv`, Q2 `IRIS.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Diabetes.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(768, 9)
```

OUTPUT

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

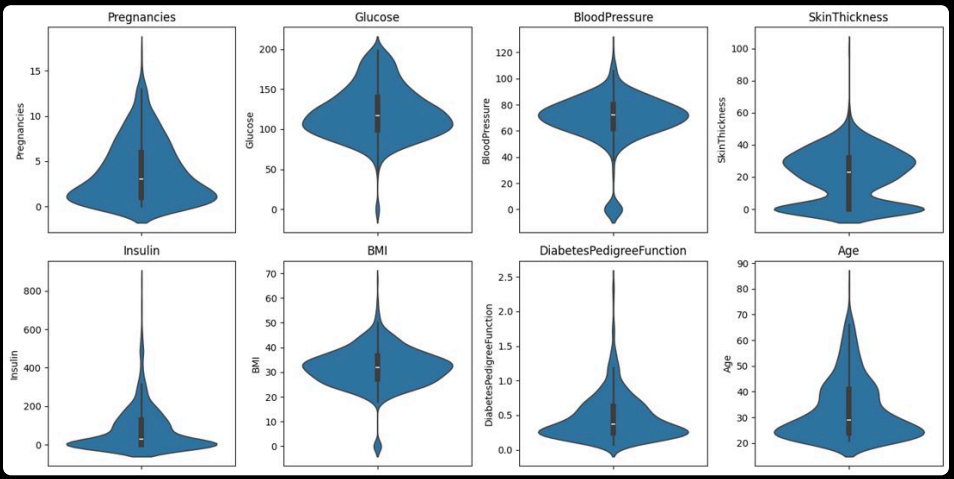
Q1.2 Display the outlier values of numerical columns using Z-score

```
num = df.select_dtypes("number")
z = ((num - num.mean()) / num.std()).abs()      # z-score: how many std-devs each
  value is from its column mean
for col in num.columns:
    outliers = num[col][z[col] > 3]
    print(f"{col}: {len(outliers)} outliers ->", outliers.values[:10])
```

```
OUTPUT
Pregnancies: 4 outliers -> [15 17 14 14]
Glucose: 5 outliers -> [0 0 0 0 0]
BloodPressure: 35 outliers -> [0 0 0 0 0 0 0 0 0 0]
SkinThickness: 1 outliers -> [99]
Insulin: 18 outliers -> [543 846 495 485 495 478 744 680 545 465]
BMI: 14 outliers -> [ 0.   0.   0.   0.   0.  67.1  0.   0.  59.4  0. ]
DiabetesPedigreeFunction: 11 outliers -> [2.288 1.893 1.781 2.329 1.476 2.137 1.731 1.6
2.42  1.699]
Age: 5 outliers -> [69 72 81 70 69]
Outcome: 0 outliers -> []
```

Q1.3 Univariate analysis of continuous variables (violin plot)

```python
fig, axes = plt.subplots(2, 4, figsize=(14, 7))
for ax, col in zip(axes.ravel(), df.columns[:8]):
    sns.violinplot(y=df[col], ax=ax)
    ax.set_title(col)
plt.tight_layout()
plt.show()
```



Q1.4 Interpolation (nearest) for numerical columns with < 10% missing

First turn the impossible zeros into missing values, so there's something to fill.

```
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[zero_is_missing] = df[zero_is_missing].replace(0, np.nan)
pct = df.isnull().mean() * 100
for col in df.columns:
    if 0 < pct[col] < 10:
        df[col] = df[col].interpolate(method="nearest").bfill().ffill()   # copy
   the nearest known value
        print(f"{col}: {pct[col]:.1f}% missing -> interpolated (nearest)")
    elif pct[col] >= 10:
        print(f"{col}: {pct[col]:.1f}% missing -> 10% or more, not interpolated")
```

OUTPUT

```
Glucose: 0.7% missing -> interpolated (nearest)
BloodPressure: 4.6% missing -> interpolated (nearest)
SkinThickness: 29.6% missing -> 10% or more, not interpolated
Insulin: 48.7% missing -> 10% or more, not interpolated
BMI: 1.4% missing -> interpolated (nearest)
```

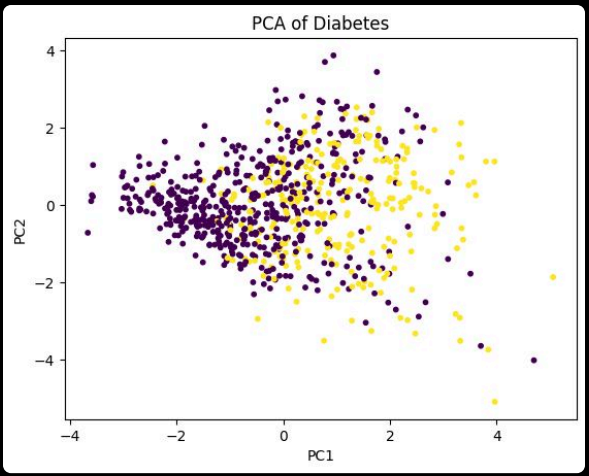
Q1.5 PCA (matrix factorisation)

```
from sklearn.preprocessing import StandardScaler
from sklearn.decomposition import PCA

X = StandardScaler().fit_transform(df.drop(columns="Outcome").fillna(df.median()))
pca = PCA(n_components=2)
pcs = pca.fit_transform(X)
print("Variance explained by PC1, PC2:", pca.explained_variance_ratio_.round(3))
plt.scatter(pcs[:, 0], pcs[:, 1], c=df["Outcome"], s=10)
plt.xlabel("PC1")
plt.ylabel("PC2")
plt.title("PCA of Diabetes")
plt.show()
```

OUTPUT

```
Variance explained by PC1, PC2: [0.284 0.187]
```



Question 2

Q2.1 Read the data and preprocess

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/IRIS.csv")
X = StandardScaler().fit_transform(df.drop(columns="Species"))   # clustering never
   sees the answer column
print(X.shape)
```

```
OUTPUT
(150, 4)
```

Q2.2 Cluster with different distance metrics (Euclidean, Manhattan) and linkages (single, complete)

Q2.4 Compare with Silhouette Score and Davies-Bouldin Index

One code cell below answers both parts.

```
from sklearn.cluster import AgglomerativeClustering
from sklearn.metrics import silhouette_score, davies_bouldin_score

rows = []
for metric in ["euclidean", "manhattan"]:
    for link in ["single", "complete"]:
        labels = AgglomerativeClustering(n_clusters=3, metric=metric,
  linkage=link).fit_predict(X)
        rows.append([metric, link, silhouette_score(X, labels),
  davies_bouldin_score(X, labels)])
results = pd.DataFrame(rows, columns=["Metric", "Linkage", "Silhouette (higher
  better)", "Davies-Bouldin (lower better)"])
results.round(4)
```

OUTPUT

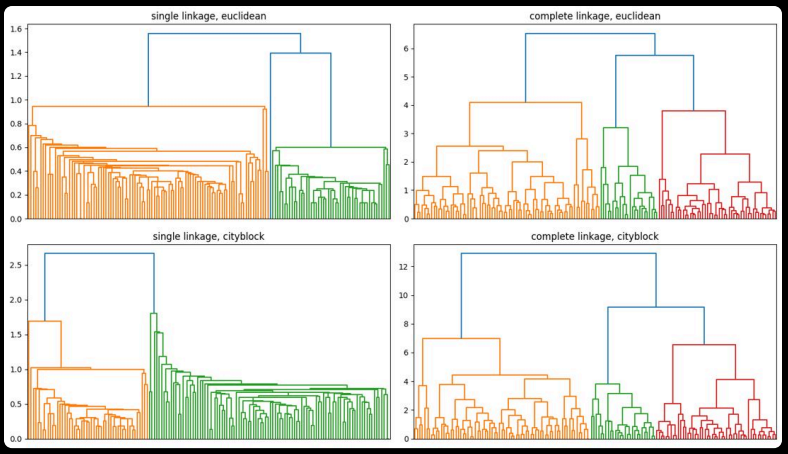
```
      Metric   Linkage  Silhouette (higher better)  \
0  euclidean    single                      0.5046
1  euclidean  complete                      0.4496
2  manhattan    single                      0.4949
3  manhattan  complete                      0.4350

   Davies-Bouldin (lower better)
0                         0.4929
1                         0.7584
2                         0.5344
3                         0.7877
```

Q2.3 Plot the dendrogram

```
import scipy.cluster.hierarchy as sch

fig, axes = plt.subplots(2, 2, figsize=(14, 8))
for ax, (metric, link) in zip(axes.ravel(), [("euclidean", "single"), ("euclidean",
  "complete"),
                                              ("cityblock", "single"),
  ("cityblock", "complete")]):
    sch.dendrogram(sch.linkage(X, method=link, metric=metric), ax=ax,
  no_labels=True)
    ax.set_title(f"{link} linkage, {metric}")
plt.tight_layout()
plt.show()
```



> In scipy, Manhattan distance is called `cityblock`. In sklearn it's `manhattan`. Same thing.

SET 9 EDA + Fuzzy C-Means

Q2 SKIP · learn Q1

Q1 → Exp 1A / 1B / 2 · Q2 → Exp 7 (Fuzzy C-Means)

Datasets: `IRIS.csv` for both.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/IRIS.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(150, 5)
```

OUTPUT

	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm	Species
0	5.1	3.5	1.4	0.2	Iris-setosa
1	4.9	3.0	1.4	0.2	Iris-setosa
2	4.7	3.2	1.3	0.2	Iris-setosa
3	4.6	3.1	1.5	0.2	Iris-setosa
4	5.0	3.6	1.4	0.2	Iris-setosa

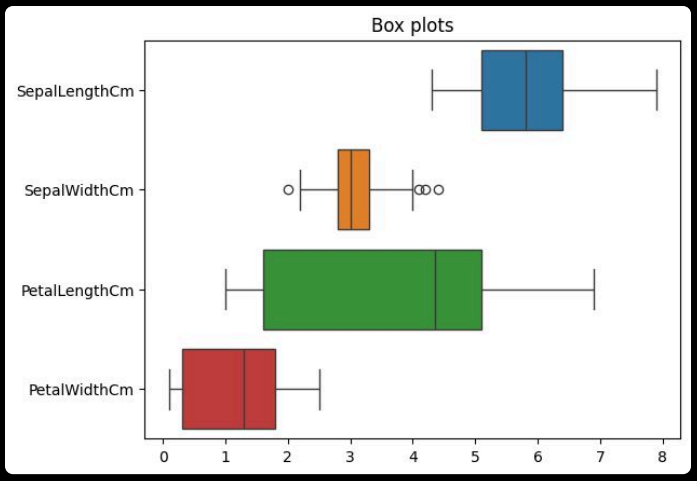
Q1.2 5-number summary, correlated with a box plot

```
num = df.select_dtypes("number")
print(num.describe().loc[["min", "25%", "50%", "75%", "max"]])   # 5-number summary
  of every numeric column

sns.boxplot(data=num, orient="h")
plt.title("Box plots")
plt.show()
```

OUTPUT

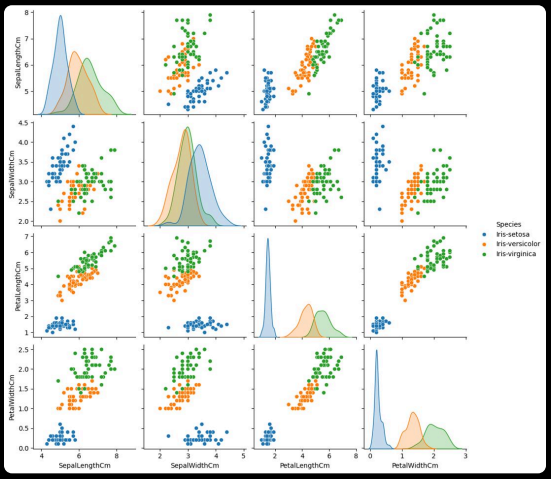
	SepalLengthCm	SepalWidthCm	PetalLengthCm	PetalWidthCm
min	4.3	2.0	1.00	0.1
25%	5.1	2.8	1.60	0.3
50%	5.8	3.0	4.35	1.3
75%	6.4	3.3	5.10	1.8
max	7.9	4.4	6.90	2.5



Correlate with the box plot: left edge of the box = 25%, middle line = 50% (median), right edge = 75%, whiskers reach towards min and max, and dots beyond the whiskers are outliers.

Q1.3 Multivariate analysis (pair plot)

```
sns.pairplot(df, hue="Species")
plt.show()
```



Q1.4 Min-max normalisation of a numerical feature (Python code)

The paper says `'Age'`, but IRIS has no Age column, so this uses `SepalLengthCm`. The formula is the same.

```
col = "SepalLengthCm"
df[col + "_minmax"] = (df[col] - df[col].min()) / (df[col].max() - df[col].min())
   # now between 0 and 1
df[[col, col + "_minmax"]].describe().loc[["min", "max"]]
```

OUTPUT

```
     SepalLengthCm  SepalLengthCm_minmax
min            4.3                   0.0
max            7.9                   1.0
```

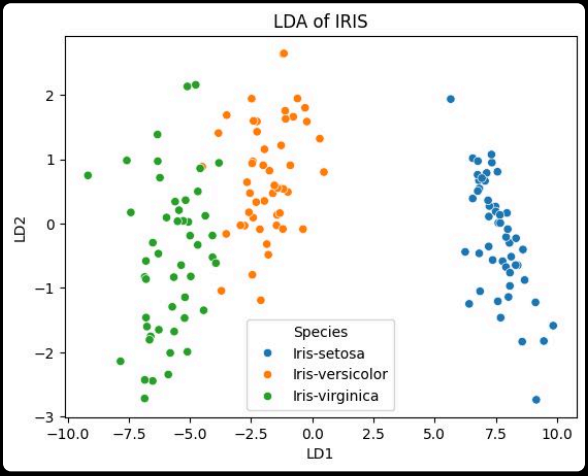
Q1.5 LDA for feature extraction

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X = df[["SepalLengthCm", "SepalWidthCm", "PetalLengthCm", "PetalWidthCm"]]
lda = LinearDiscriminantAnalysis(n_components=2)
ld = lda.fit_transform(X, df["Species"])            # LDA uses the labels (PCA
   doesn't)
print("Explained variance:", lda.explained_variance_ratio_.round(3))
sns.scatterplot(x=ld[:, 0], y=ld[:, 1], hue=df["Species"])
plt.xlabel("LD1")
plt.ylabel("LD2")
plt.title("LDA of IRIS")
plt.show()
```

OUTPUT

```
Explained variance: [0.991 0.009]
```



Question 2

Q2.1 Read the data and preprocess

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/IRIS.csv")
X = StandardScaler().fit_transform(df.drop(columns="Species"))   # clustering never
   sees the answer column
print(X.shape)
```

```
OUTPUT
(150, 4)
```

The FCM function (the same as Exp7_SHORT)

```
def fcm(X, c, m=2):
    U = np.random.default_rng(42).random((c, len(X)))
    U = U / U.sum(axis=0)                                          # every column
   sums to 1
    for it in range(1, 301):
        U_old = U.copy()
        Um = U ** m
        centroids = (Um @ X) / Um.sum(axis=1, keepdims=True)         # weighted
   average of the points
        d = np.linalg.norm(X[None] - centroids[:, None], axis=2)     # distance of
   every point to every centroid
        U = 1 / np.sum((d[:, None] / d[None]) ** (2 / (m - 1)), axis=1)
        if np.linalg.norm(U - U_old) < 1e-5:
            break
    return centroids, U

def pc_score(U):
    return np.sum(U ** 2) / U.shape[1]            # partition coefficient: higher =
   better

def ce_score(U):
    return -np.sum(U * np.log(U)) / U.shape[1]    # classification entropy: lower =
   better
```

Q2.2 Determine the optimal number of clusters

Q2.4 Measure performance with Partition Coefficient and Classification Entropy

One code cell below answers both parts.

```
rows = []
for c in range(2, 7):
    centroids, U = fcm(X, c)
    rows.append([c, pc_score(U), ce_score(U)])
scores = pd.DataFrame(rows, columns=["c", "PC (higher better)", "CE (lower
  better)"])
print(scores.round(4))
best = int(scores.loc[scores["PC (higher better)"].idxmax(), "c"])
print("Optimal number of clusters:", best)
```

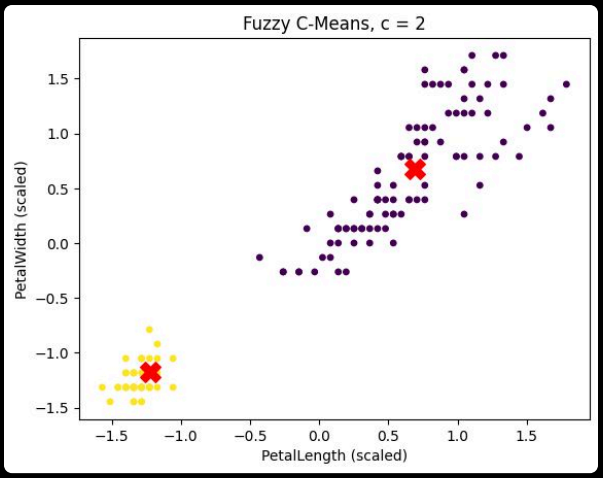
```
OUTPUT
   c  PC (higher better)  CE (lower better)
0  2              0.8339             0.2869
1  3              0.7065             0.5294
2  4              0.6341             0.6902
3  5              0.5700             0.8398
4  6              0.5117             0.9877
Optimal number of clusters: 2
```

Q2.3 Print and plot the cluster centroids and cluster labels

```
centroids, U = fcm(X, best)
labels = U.argmax(axis=0)               # each point goes to the cluster it belongs
  to MOST
print("Centroids:\n", centroids.round(3))
print("Labels:", labels)

plt.scatter(X[:, 2], X[:, 3], c=labels, s=15)
plt.scatter(centroids[:, 2], centroids[:, 3], c="red", marker="X", s=200)
plt.xlabel("PetalLength (scaled)")
plt.ylabel("PetalWidth (scaled)")
plt.title(f"Fuzzy C-Means, c = {best}")
plt.show()
```

```
OUTPUT
Centroids:
 [[ 0.579 -0.364  0.692  0.673]
 [-0.974  0.741 -1.218 -1.175]]
Labels: [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0]
```



Viva: IRIS has 3 species, but PC picks **2** clusters, because setosa sits far away while versicolor and virginica overlap. PC says "2 clean groups" beats "3 fuzzy groups".

SET 10 EDA + SOM Clustering

Q2 SKIP · learn Q1

Q1 → Exp 1A / 1B · Q2 → Exp 8 (SOM)

Datasets: Q1 `Solar.csv`, Q2 `IRIS.csv`. The SOM needs `pip install sklearn-som` if a lab PC doesn't have it.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Solar.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(228, 7)
```

OUTPUT

```
   Latitude  Longitude  Altitude  min Temo  Max Temp  Sunshine Hour  \
0     34.08      74.79      1730      -0.9       4.3       0.450000
1     34.08      74.79      1730      -0.2       7.6       0.464286
2     34.08      74.79      1730       2.6      14.2       0.521429
3     34.08      74.79      1730       1.8       8.1       0.542857
4     34.08      74.79      1730       7.5      20.4       0.628571

   Solar Radiation
0                0
1                0
2                0
3                0
4                0
```

Q1.2 5-number summary of the numerical columns, correlated with a box plot

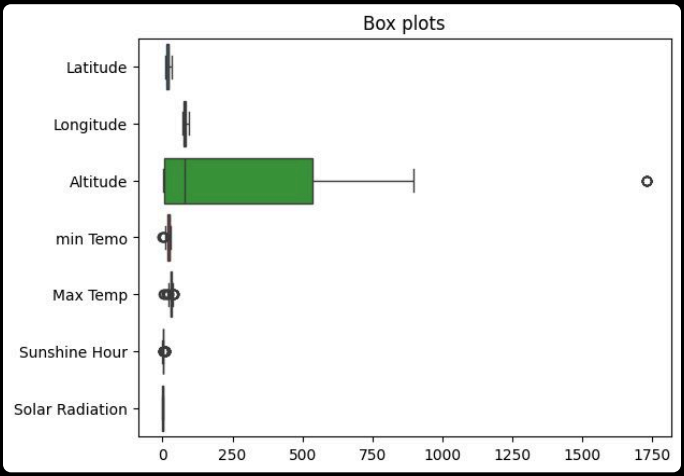
```
num = df.select_dtypes("number")
print(num.describe().loc[["min", "25%", "50%", "75%", "max"]])   # 5-number summary
   of every numeric column

sns.boxplot(data=num, orient="h")
plt.title("Box plots")
plt.show()
```

OUTPUT

```
     Latitude  Longitude  Altitude  min Temo  Max Temp  Sunshine Hour  \
min      8.28      72.15       2.0      -0.9       4.3       0.428571
25%     15.49      73.81       7.0      19.0      27.9       0.605357
50%     21.09      77.42      81.0      23.8      29.9       0.678571
75%     23.35      83.14     536.0      26.2      32.0       0.742857
max     34.08      92.72    1730.0      29.7      41.1      11.000000

     Solar Radiation
min              0.0
25%              0.0
50%              0.5
75%              1.0
max              1.0
```



Correlate with the box plot: left edge of the box = 25%, middle line = 50% (median), right edge = 75%, whiskers reach towards min and max, and dots beyond the whiskers are outliers. Altitude has much bigger numbers than the rest, which is exactly why Q1.3 scales the data.

Q1.3 Scaling with StandardScaler

```
from sklearn.preprocessing import StandardScaler

num = df.select_dtypes("number")
scaled = pd.DataFrame(StandardScaler().fit_transform(num), columns=num.columns)
print(scaled.describe().loc[["mean", "std"]].round(2))      # every column now has
   mean 0, std 1
scaled.head()
```

OUTPUT

```
      Latitude  Longitude  Altitude  min Temo  Max Temp  Sunshine Hour  \
mean       0.0       -0.0      -0.0       0.0      -0.0            0.0
std        1.0        1.0       1.0       1.0       1.0            1.0

      Solar Radiation
mean              0.0
std               1.0
```

OUTPUT

```
   Latitude  Longitude  Altitude  min Temo  Max Temp  Sunshine Hour  \
0  2.322946  -0.701343  3.294297 -3.998271 -5.303794      -0.488058
1  2.322946  -0.701343  3.294297 -3.875781 -4.617952      -0.482631
2  2.322946  -0.701343  3.294297 -3.385821 -3.246267      -0.460919
3  2.322946  -0.701343  3.294297 -3.525810 -4.514036      -0.452777
4  2.322946  -0.701343  3.294297 -2.528391 -1.957714      -0.420210

   Solar Radiation
0             -1.0
1             -1.0
2             -1.0
3             -1.0
4             -1.0
```

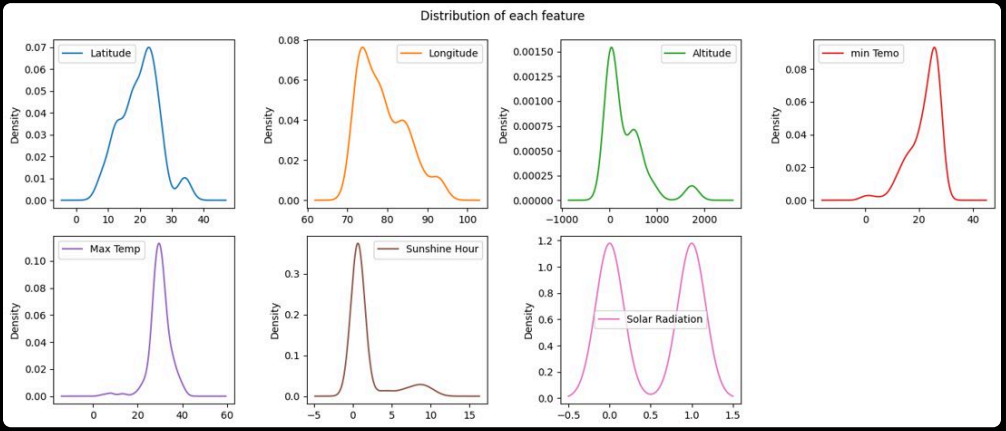
Q1.4 Skewness with distribution plots: right, left and no skew

```
num = df.select_dtypes("number")
skew = num.skew()
print(skew.round(2))
print("\nRight skew (> 0.5):", list(skew[skew > 0.5].index))
print("Left skew (< -0.5):", list(skew[skew < -0.5].index))
print("No skew  (-0.5 to 0.5):", list(skew[skew.abs() <= 0.5].index))

num.plot(kind="kde", subplots=True, layout=(-1, 4), figsize=(14, 6), sharex=False,
   title="Distribution of each feature")
plt.tight_layout()
plt.show()
```

```
OUTPUT
Latitude          0.11
Longitude         0.75
Altitude          1.89
min Temo         -1.40
Max Temp         -1.59
Sunshine Hour     2.24
Solar Radiation   0.00
dtype: float64

Right skew (> 0.5): ['Longitude', 'Altitude', 'Sunshine Hour']
Left skew (< -0.5): ['min Temo', 'Max Temp']
No skew  (-0.5 to 0.5): ['Latitude', 'Solar Radiation']
```



> Right skew = long tail on the right (mean > median). Left skew = long tail on the left (mean < median).

Q1.5 Z-score normalisation of a numerical feature (Python code)

```python
col = "Max Temp"
df[col + "_z"] = (df[col] - df[col].mean()) / df[col].std()     # z = (x - mean) / std
df[[col, col + "_z"]].head()
```

```
OUTPUT
   Max Temp  Max Temp_z
0       4.3   -5.292150
1       7.6   -4.607813
2      14.2   -3.239140
3       8.1   -4.504126
4      20.4   -1.953416
```

Question 2

Q2.1 Read the data and preprocess

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/IRIS.csv")
X = StandardScaler().fit_transform(df.drop(columns="Species"))   # clustering never
   sees the answer column
print(X.shape)
```

OUTPUT

```
(150, 4)
```

Q2.2 SOM clustering into N clusters

```
from sklearn_som.som import SOM

n = 3                                    # N = number of clusters we want
som = SOM(m=1, n=n, dim=X.shape[1], lr=1, sigma=0.5, random_state=42)   # a 1 x n
   grid = n clusters
som.fit(X, epochs=50, shuffle=True)      # train the map
labels = som.predict(X)                  # cluster number of every row
print("Points in each cluster:", np.bincount(labels))
```

OUTPUT

```
Points in each cluster: [44 49 57]
```

A SOM with a **1 x n** grid has n neurons, and every point goes to its closest neuron, so we get **n clusters**.

Q2.3 Measure performance with Silhouette and DB score

```
from sklearn.metrics import silhouette_score, davies_bouldin_score

print("Silhouette (higher better):", round(silhouette_score(X, labels), 4))
print("DB score   (lower better) :", round(davies_bouldin_score(X, labels), 4))

# try other values of n and compare
rows = []
for k in range(2, 7):
    s = SOM(m=1, n=k, dim=X.shape[1], lr=1, sigma=0.5, random_state=42)
    s.fit(X, epochs=50, shuffle=True)
    lab = s.predict(X)
    rows.append([k, silhouette_score(X, lab), davies_bouldin_score(X, lab)])
scores = pd.DataFrame(rows, columns=["n", "Silhouette (higher better)", "DB (lower
   better)"])
scores.round(4)
```

```
OUTPUT
Silhouette (higher better): 0.456
DB score   (lower better) : 0.8256
```

```
OUTPUT
   n  Silhouette (higher better)  DB (lower better)
0  2                      0.5818             0.5933
1  3                      0.4560             0.8256
2  4                      0.4127             0.9235
3  5                      0.3825             1.0051
4  6                      0.3266             1.0508
```

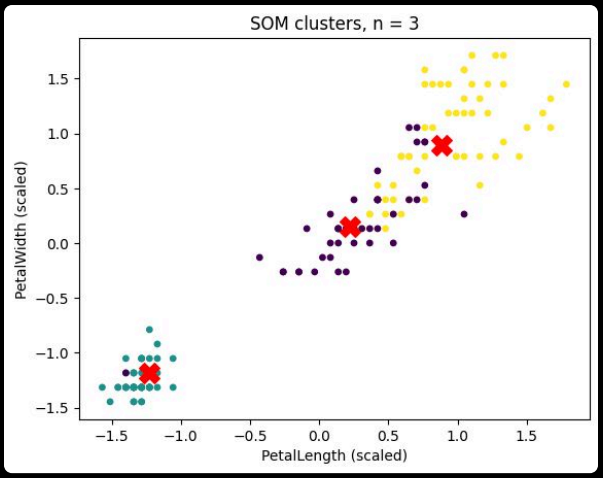
> n = 2 gets the best scores because setosa is far away and the other two species overlap. n = 3 matches the 3 real species, so we keep n = 3 for the plot.

Q2.4 Print and plot the cluster centroids and labels for n clusters

```
centroids = som.weights                 # each neuron's weights = that cluster's
   centroid
print("Centroids:\n", centroids.round(3))
print("Labels:", labels)

plt.scatter(X[:, 2], X[:, 3], c=labels, s=15)
plt.scatter(centroids[:, 2], centroids[:, 3], c="red", marker="X", s=200)
plt.xlabel("PetalLength (scaled)")
plt.ylabel("PetalWidth (scaled)")
plt.title(f"SOM clusters, n = {n}")
plt.show()
```

```
OUTPUT
Centroids:
 [[-0.223 -0.973  0.224  0.148]
 [-0.952  0.847 -1.232 -1.187]
 [ 0.985 -0.004  0.884  0.897]]
Labels: [1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 0 1 1 1 1 1 1 1 2 2 2 0 2 0 2 0 2 0 0 0 0 0 0 2 0 0 0 0 2 0 0 0
 2 2 2 2 0 0 0 0 0 0 0 2 2 0 0 0 0 2 0 0 0 0 0 0 0 0 2 0 2 2 2 2 0 2 2 2 2
 2 2 0 2 2 2 2 2 0 2 0 2 2 2 2 2 2 2 2 2 2 2 0 2 2 2 2 2 2 2 0 2 2 2 0 2
 2 2]
```


SOM clusters, n = 3
PetalWidth (scaled)
PetalLength (scaled)
1.5
1.0
0.5
0.0
−0.5
−1.0
−1.5
−1.5
−1.0
−0.5
0.0
0.5
1.0
1.5

SET 11 EDA + Apriori

Q2 SKIP · learn Q1

Q1 → Exp 1A / 2 · Q2 → Exp 9 (Apriori)

Datasets: Q1 `Diabetes.csv`. Q2 asks for `Supermarket.csv`, which we don't have, so this uses `Shop1.csv` (your Exp9 shopping baskets). The thresholds are the paper's: **support 10%, confidence 70%**.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
   clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Diabetes.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(768, 9)
```

OUTPUT

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Q1.2 Display the columns with null values

```
nulls = df.isnull().sum()
print(nulls)
print("\nColumns with null values:", list(nulls[nulls > 0].index))
```

```
OUTPUT
Pregnancies                 0
Glucose                     0
BloodPressure               0
SkinThickness               0
Insulin                     0
BMI                         0
DiabetesPedigreeFunction    0
Age                         0
Outcome                     0
dtype: int64

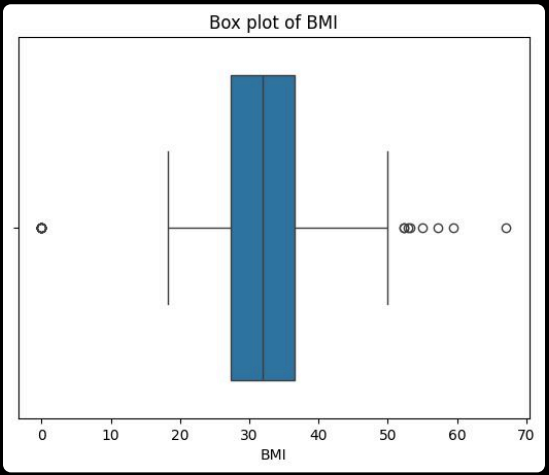
Columns with null values: []
```

Q1.3 5-number summary, correlated with a box plot

```
col = "BMI"
print(df[col].describe()[["min", "25%", "50%", "75%", "max"]])     # the 5-number
   summary

sns.boxplot(x=df[col])
plt.title("Box plot of " + col)
plt.show()
```

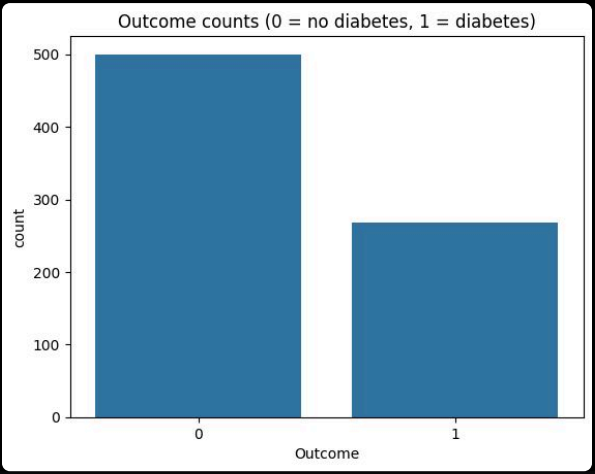
```
OUTPUT
min     0.0
25%    27.3
50%    32.0
75%    36.6
max    67.1
Name: BMI, dtype: float64
```



Correlate with the box plot: left edge of the box = 25%, middle line = 50% (median), right edge = 75%, whiskers reach towards min and max, and dots beyond the whiskers are outliers.

Q1.4 Counts of a categorical variable (count plot)

```
sns.countplot(x="Outcome", data=df)
plt.title("Outcome counts (0 = no diabetes, 1 = diabetes)")
plt.show()
print(df["Outcome"].value_counts())
```




```
OUTPUT
Outcome
0    500
1    268
Name: count, dtype: int64
```

Q1.5 Univariate feature selection (SelectKBest, SelectPercentile)

```
from sklearn.feature_selection import SelectKBest, SelectPercentile, f_classif

X = df.drop(columns="Outcome")
y = df["Outcome"]
kbest = SelectKBest(f_classif, k=4).fit(X, y)
print("SelectKBest (k=4):", list(kbest.get_feature_names_out()))
perc = SelectPercentile(f_classif, percentile=50).fit(X, y)
print("SelectPercentile (50%):", list(perc.get_feature_names_out()))
print(pd.Series(kbest.scores_,
   index=X.columns).sort_values(ascending=False).round(1))
```

```
OUTPUT
SelectKBest (k=4): ['Pregnancies', 'Glucose', 'BMI', 'Age']
SelectPercentile (50%): ['Pregnancies', 'Glucose', 'BMI', 'Age']
Glucose                     213.2
BMI                          71.8
Age                          46.1
Pregnancies                  39.7
DiabetesPedigreeFunction     23.9
Insulin                      13.3
SkinThickness                 4.3
BloodPressure                 3.3
dtype: float64
```

Question 2

Q2.1 Read the data

```
from mlxtend.preprocessing import TransactionEncoder

shop = pd.read_csv("data/Shop1.csv", header=None)
transactions = [[item.strip() for item in row.dropna()] for _, row in
   shop.iterrows()]   # one list per basket
te = TransactionEncoder()
basket = pd.DataFrame(te.fit(transactions).transform(transactions),
   columns=te.columns_)   # True/False table
print(len(transactions), "transactions,", basket.shape[1], "items")
basket.head()
```

```
OUTPUT
250 transactions, 22 items
```

```
OUTPUT
   Apples  Bananas   Beer  Biscuits  Bread  Butter  Cheese  Chips  Coffee  \
0   False    False  False     False  False   False    True  False   False
1   False    False  False     False   True   False   False  False   False
2   False     True  False     False   True    True    True  False   False
3   False    False  False     False  False   False   False   True   False
4    True    False  False     False  False   False   False  False    True

    Cola  ...   Eggs    Jam   Milk    Oil  Onions   Rice  Sugar    Tea  \
0  False  ...  False  False  False  False   False  False  False  False
1  False  ...   True  False   True  False   False  False  False  False
2  False  ...  False  False  False  False   False  False  False  False
3   True  ...  False  False  False  False   False  False   True   True
4  False  ...  False  False  False  False   False  False   True  False

   Tomatoes  Yogurt
0     False   False
... (6 more lines)
```

Q2.2 Display the candidate set (C) and frequent set (L) for every iteration

This is Apriori written out step by step: count every candidate, keep those with support ≥ 10%, then join the survivors to make the next, bigger candidates.

```
MIN_SUPPORT = 0.10
candidates = [frozenset([item]) for item in basket.columns]
k = 1
while candidates:
    support = {c: basket[list(c)].all(axis=1).mean() for c in candidates}   # % of
   baskets holding ALL items of c
    frequent = [c for c in candidates if support[c] >= MIN_SUPPORT]
    print(f"Iteration {k}:  C{k} = {len(candidates)} candidates,  L{k} =
   {len(frequent)} frequent")
    for c in frequent:
        print("   ", sorted(c), round(support[c], 3))
    k += 1
    candidates = list({a | b for a in frequent for b in frequent if len(a | b) ==
   k})   # join step
```

OUTPUT

```
Iteration 1:  C1 = 22 candidates,  L1 = 16 frequent
    ['Apples'] 0.108
    ['Biscuits'] 0.168
    ['Bread'] 0.448
    ['Butter'] 0.368
    ['Chips'] 0.236
    ['Coffee'] 0.18
    ['Cola'] 0.216
    ['Dal'] 0.224
    ['Eggs'] 0.212
    ['Jam'] 0.18
    ['Milk'] 0.3
    ['Oil'] 0.168
    ['Rice'] 0.3
    ['Sugar'] 0.308
    ['Tea'] 0.252
... (27 more lines)
```

Q2.3 Display the association rules (confidence ≥ 70%)

```
from mlxtend.frequent_patterns import apriori, association_rules

itemsets = apriori(basket, min_support=MIN_SUPPORT, use_colnames=True)
rules = association_rules(itemsets, metric="confidence", min_threshold=0.70)
rules[["antecedents", "consequents", "support", "confidence", "lift"]].round(3)
```

```
OUTPUT
       antecedents consequents  support  confidence   lift
0         (Butter)     (Bread)    0.312       0.848  1.892
1          (Cola)     (Chips)    0.164       0.759  3.217
2           (Dal)      (Rice)    0.224       1.000  3.333
3          (Rice)       (Dal)    0.224       0.747  3.333
4           (Tea)     (Sugar)    0.184       0.730  2.371
5     (Jam, Bread)    (Butter)    0.108       0.871  2.367
6    (Jam, Butter)     (Bread)    0.108       1.000  2.232
7   (Milk, Butter)     (Bread)    0.100       0.962  2.146
8     (Dal, Bread)      (Rice)    0.104       1.000  3.333
9    (Rice, Bread)       (Dal)    0.104       0.743  3.316
10      (Dal, Oil)      (Rice)    0.104       1.000  3.333
11     (Rice, Oil)       (Dal)    0.104       0.963  4.299
```

Q2.4 Rules with the highest confidence

```
rules.sort_values("confidence", ascending=False)[["antecedents", "consequents",
  "confidence", "lift"]].head(5).round(3)
```

```
OUTPUT
      antecedents consequents  confidence   lift
2           (Dal)      (Rice)       1.000  3.333
6   (Jam, Butter)     (Bread)       1.000  2.232
8    (Dal, Bread)      (Rice)       1.000  3.333
10     (Dal, Oil)      (Rice)       1.000  3.333
11    (Rice, Oil)       (Dal)       0.963  4.299
```

Support = how often the items appear together. Confidence = when A is bought, how often B is also bought. Lift > 1 = A really boosts B.

SET 12 EDA + Random Forest (4 models)

LEARN Q2

Q1 → Exp 1A / 1B · Q2 → Exp 10 (Random Forest)

Datasets: Q1 `Diabetes.csv`. Q2 asks for `house_price.csv`, which we don't have, so this uses `Diabetes.csv`. If your house file has a numeric price, make 2 classes first: `df["Class"] = (df["Price"] > df["Price"].median()).astype(int)`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Diabetes.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(768, 9)
```

OUTPUT

	Pregnancies	Glucose	BloodPressure	SkinThickness	Insulin	BMI	\
0	6	148	72	35	0	33.6	
1	1	85	66	29	0	26.6	
2	8	183	64	0	0	23.3	
3	1	89	66	23	94	28.1	
4	0	137	40	35	168	43.1	

	DiabetesPedigreeFunction	Age	Outcome
0	0.627	50	1
1	0.351	31	0
2	0.672	32	1
3	0.167	21	0
4	2.288	33	1

Q1.2 Display the outlier values of numerical columns using Z-score

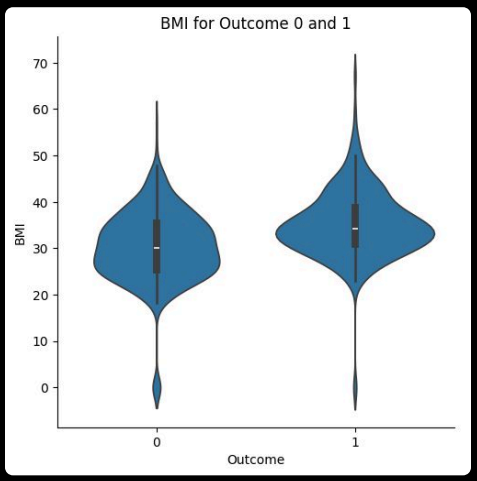
```
num = df.select_dtypes("number")
z = ((num - num.mean()) / num.std()).abs()      # z-score: how many std-devs each
   value is from its column mean
for col in num.columns:
    outliers = num[col][z[col] > 3]
    print(f"{col}: {len(outliers)} outliers ->", outliers.values[:10])
```

OUTPUT

```
Pregnancies: 4 outliers -> [15 17 14 14]
Glucose: 5 outliers -> [0 0 0 0 0]
BloodPressure: 35 outliers -> [0 0 0 0 0 0 0 0 0 0]
SkinThickness: 1 outliers -> [99]
Insulin: 18 outliers -> [543 846 495 485 495 478 744 680 545 465]
BMI: 14 outliers -> [ 0.   0.   0.   0.   0.  67.1  0.   0.  59.4  0. ]
DiabetesPedigreeFunction: 11 outliers -> [2.288 1.893 1.781 2.329 1.476 2.137 1.731 1.6
2.42  1.699]
Age: 5 outliers -> [69 72 81 70 69]
Outcome: 0 outliers -> []
```

Q1.3 Bivariate analysis: continuous vs categorical (categorical plot)

```
sns.catplot(x="Outcome", y="BMI", data=df, kind="violin")
plt.title("BMI for Outcome 0 and 1")
plt.show()
```



Q1.4 Mode imputation for categorical data if missing > 10%

```
pct = df.isnull().mean() * 100                    # % missing in every column
for col in ["Outcome"]:
    if pct[col] > 10:
        df[col] = df[col].fillna(df[col].mode()[0])
        print(f"{col}: {pct[col]:.1f}% missing -> filled with mode")
    else:
        print(f"{col}: {pct[col]:.1f}% missing -> not more than 10%, left as it
  is")
```

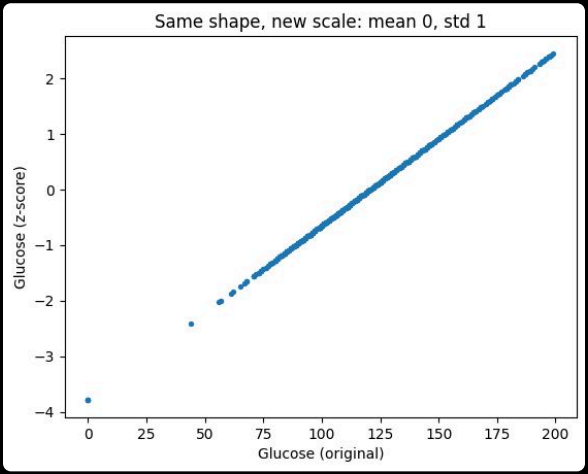
OUTPUT

```
Outcome: 0.0% missing -> not more than 10%, left as it is
```

Outcome is the only categorical column, and it has no missing values. The same loop fills any categorical column that's above 10%.

Q1.5 Z-score normalisation of a numerical feature, analysed with a scatter plot

```
col = "Glucose"
df[col + "_z"] = (df[col] - df[col].mean()) / df[col].std()
plt.scatter(df[col], df[col + "_z"], s=8)
plt.xlabel(col + " (original)")
plt.ylabel(col + " (z-score)")
plt.title("Same shape, new scale: mean 0, std 1")
plt.show()
```



Question 2

Q2.1 Read the dataset and preprocess

Q2.2 Choose X (independent) and Y (dependent)

One code cell below answers both parts.

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/Diabetes.csv")
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
   # a 0 here is impossible = missing
df[zero_is_missing] = df[zero_is_missing].replace(0, np.nan)
df = df.fillna(df.median())                              # fill with the median
X = StandardScaler().fit_transform(df.drop(columns="Outcome"))
y = df["Outcome"]
print(X.shape, y.value_counts().to_dict())

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
(768, 8) {0: 500, 1: 268}
Train: (576, 8)  Test: (192, 8)
```

Q2.3 Four Random Forest models with different parameters

Q2.5 Confusion table, accuracy, precision, recall and F-score for each

One code cell below answers both parts.

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

params = [dict(n_estimators=10, max_depth=None), dict(n_estimators=50,
   max_depth=5),
          dict(n_estimators=100, criterion="entropy"), dict(n_estimators=200,
   max_depth=8, min_samples_leaf=2)]
rows = []
for p in params:
    pred = RandomForestClassifier(random_state=42, **p).fit(X_train,
   y_train).predict(X_test)
    print(p, "\n", confusion_matrix(y_test, pred))
    rows.append([str(p), accuracy_score(y_test, pred), precision_score(y_test,
   pred, average="macro"),
                 recall_score(y_test, pred, average="macro"), f1_score(y_test,
   pred, average="macro")])
pd.DataFrame(rows, columns=["Parameters", "Accuracy", "Precision", "Recall", "F-
   score"]).round(4)
```

OUTPUT

```
{'n_estimators': 10, 'max_depth': None}
 [[94 29]
 [28 41]]
{'n_estimators': 50, 'max_depth': 5}
 [[98 25]
 [25 44]]
{'n_estimators': 100, 'criterion': 'entropy'}
 [[97 26]
 [21 48]]
{'n_estimators': 200, 'max_depth': 8, 'min_samples_leaf': 2}
 [[100  23]
 [ 22  47]]
```

OUTPUT

	Parameters	Accuracy	Precision	Recall	F-score
0	{'n_estimators': 10, 'max_depth': None}	0.7031	0.6781	0.6792	0.6786
1	{'n_estimators': 50, 'max_depth': 5}	0.7396	0.7172	0.7172	0.7172
2	{'n_estimators': 100, 'criterion': 'entropy'}	0.7552	0.7353	0.7421	0.7382
3	{'n_estimators': 200, 'max_depth': 8, 'min_sam...	0.7656	0.7456	0.7471	0.7463

Q2.4 Find the best parameters and calculate Y_predict

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(RandomForestClassifier(random_state=42),
                      {"n_estimators": [50, 100, 200], "max_depth": [None, 5, 8]},
   cv=5, n_jobs=-1).fit(X_train, y_train)
print("Best parameters:", search.best_params_)
y_pred = search.predict(X_test)
print("Y_predict:", y_pred)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
```

```
OUTPUT
Best parameters: {'max_depth': 8, 'n_estimators': 50}
Y_predict: [1 0 0 0 1 1 0 1 1 1 0 1 0 0 0 0 0 0 1 1 0 0 0 0 0 1 0 0 0 0 1 1 1 1 1 1 1
 0 0 0 0 0 1 0 0 1 1 0 0 1 0 1 1 0 0 0 1 0 0 1 1 0 0 0 0 1 0 1 0 1 1 0 0 0
 0 0 0 0 0 0 1 0 0 1 0 1 1 0 0 0 0 0 0 0 0 1 1 0 1 0 1 0 1 1 1 0 0 1 0 1 0
 1 0 1 0 0 1 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 0 0 1 0 0 1 1 0 0 0 0 1 0 0 0 0
 0 1 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 0 1 0 0 1 1 1 0 1 0 1 0 0 0 0 1 0 0 0 1 1
 0 0 0 1 1 0 0]
Accuracy: 0.7396
```

SET 13 EDA + AdaBoost (4 models)

LEARN Q2

Q1 → Exp 1B / 2 · Q2 → Exp 10 (AdaBoost)

Datasets: Q1 `Diabetes.csv`, Q2 `IRIS.csv` (3 species, so precision / recall use `average="macro"`).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Diabetes.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(768, 9)
```

OUTPUT

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Q1.2 % of missing values per column, and drop the column if > 20%

The impossible zeros are the real missing values here, so turn them into NaN first.

```python
zero_is_missing = ["Glucose", "BloodPressure", "SkinThickness", "Insulin", "BMI"]
df[zero_is_missing] = df[zero_is_missing].replace(0, np.nan)
pct = df.isnull().mean() * 100
print(pct.round(1))
drop_cols = pct[pct > 20].index
print("\nDropping (> 20% missing):", list(drop_cols))
df = df.drop(columns=drop_cols)
```

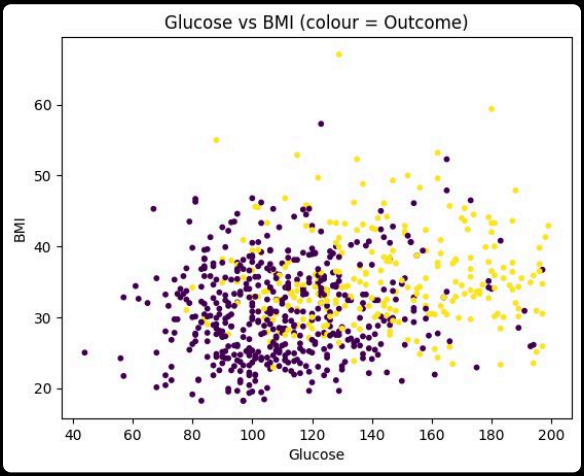
OUTPUT

```
Pregnancies                 0.0
Glucose                     0.7
BloodPressure               4.6
SkinThickness              29.6
Insulin                    48.7
BMI                         1.4
DiabetesPedigreeFunction    0.0
Age                         0.0
Outcome                     0.0
dtype: float64

Dropping (> 20% missing): ['SkinThickness', 'Insulin']
```

Q1.3 Scatter plot of two continuous variables

```python
plt.scatter(df["Glucose"], df["BMI"], c=df["Outcome"], s=10)
plt.xlabel("Glucose")
plt.ylabel("BMI")
plt.title("Glucose vs BMI (colour = Outcome)")
plt.show()
```



Q1.4 Label encoding of a categorical feature

Diabetes has no text column, so we make one (age groups) and encode it.

```
df["AgeGroup"] = pd.cut(df["Age"], bins=[20, 30, 40, 50, 90], labels=["20s", "30s",
   "40s", "50+"])
from sklearn.preprocessing import LabelEncoder

print("Before:", df["AgeGroup"].unique()[:6])
df["AgeGroup"] = LabelEncoder().fit_transform(df["AgeGroup"].astype(str))
print("After :", df["AgeGroup"].unique()[:6])
```

OUTPUT

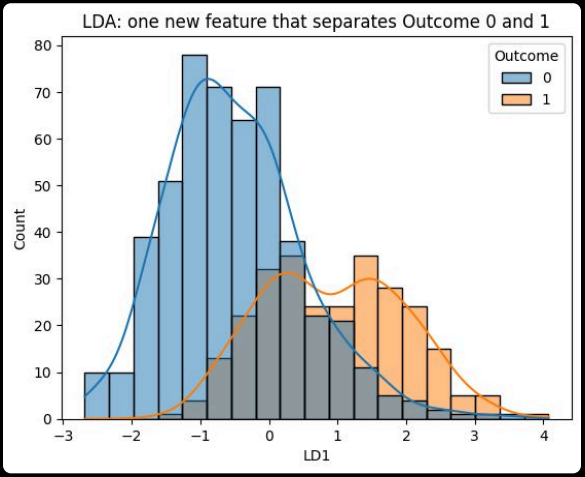
```
Before: ['40s', '30s', '20s', '50+']
Categories (4, object): ['20s' < '30s' < '40s' < '50+']
After : [2 1 0 3]
```

Q1.5 LDA for feature extraction

With 2 classes, LDA can make at most **1** new feature (classes − 1).

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

X = df.drop(columns=["Outcome", "AgeGroup"]).fillna(df.median(numeric_only=True))
ld = LinearDiscriminantAnalysis(n_components=1).fit_transform(X, df["Outcome"])
sns.histplot(x=ld[:, 0], hue=df["Outcome"], kde=True)
plt.xlabel("LD1")
plt.title("LDA: one new feature that separates Outcome 0 and 1")
plt.show()
```



Question 2

Q2.1 Read the dataset and preprocess

Q2.2 Choose X (independent) and Y (dependent)

One code cell below answers both parts.

```
df = pd.read_csv("data/IRIS.csv")
X = df.drop(columns="Species")
y = df["Species"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
   random_state=42, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
Train: (112, 4)  Test: (38, 4)
```

Q2.3 Four AdaBoost models with different parameters

Q2.5 Confusion table, accuracy, precision, recall and F-score for each

One code cell below answers both parts.

```
from sklearn.ensemble import AdaBoostClassifier
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

params = [dict(n_estimators=10, learning_rate=1.0), dict(n_estimators=50,
   learning_rate=0.5),
          dict(n_estimators=100, learning_rate=0.1), dict(n_estimators=200,
   learning_rate=0.05)]
rows = []
for p in params:
    pred = AdaBoostClassifier(random_state=42, **p).fit(X_train,
   y_train).predict(X_test)
    print(p, "\n", confusion_matrix(y_test, pred))
    rows.append([str(p), accuracy_score(y_test, pred), precision_score(y_test,
   pred, average="macro"),
                 recall_score(y_test, pred, average="macro"), f1_score(y_test,
   pred, average="macro")])
pd.DataFrame(rows, columns=["Parameters", "Accuracy", "Precision", "Recall", "F-
   score"]).round(4)
```

```
OUTPUT
{'n_estimators': 10, 'learning_rate': 1.0}
 [[12  0  0]
 [ 0 12  1]
 [ 0  2 11]]
```

```
OUTPUT
{'n_estimators': 50, 'learning_rate': 0.5}
 [[12  0  0]
 [ 0 12  1]
 [ 0  3 10]]
{'n_estimators': 100, 'learning_rate': 0.1}
 [[12  0  0]
 [ 0 12  1]
 [ 0  1 12]]
```

```
OUTPUT
{'n_estimators': 200, 'learning_rate': 0.05}
 [[12  0  0]
 [ 0 12  1]
 [ 0  1 12]]
```

```
OUTPUT
                                  Parameters  Accuracy  Precision  Recall  \
0    {'n_estimators': 10, 'learning_rate': 1.0}    0.9211     0.9246  0.9231
1    {'n_estimators': 50, 'learning_rate': 0.5}    0.8947     0.9030  0.8974
2   {'n_estimators': 100, 'learning_rate': 0.1}    0.9474     0.9487  0.9487
3  {'n_estimators': 200, 'learning_rate': 0.05}    0.9474     0.9487  0.9487

   F-score
0   0.9230
1   0.8968
2   0.9487
3   0.9487
```

Q2.4 Find the best parameters and calculate Y_predict

```
from sklearn.model_selection import GridSearchCV

search = GridSearchCV(AdaBoostClassifier(random_state=42),
                      {"n_estimators": [10, 50, 100, 200], "learning_rate": [0.05,
   0.1, 0.5, 1.0]}, cv=5, n_jobs=-1).fit(X_train, y_train)
print("Best parameters:", search.best_params_)
y_pred = search.predict(X_test)
print("Y_predict:", y_pred)
print("Accuracy:", round(accuracy_score(y_test, y_pred), 4))
```

```
OUTPUT
Best parameters: {'learning_rate': 0.5, 'n_estimators': 10}
Y_predict: ['Iris-setosa' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-setosa' 'Iris-versicolor' 'Iris-virginica' 'Iris-virginica'
 'Iris-virginica' 'Iris-virginica' 'Iris-versicolor' 'Iris-virginica'
 'Iris-versicolor' 'Iris-versicolor' 'Iris-setosa' 'Iris-setosa'
 'Iris-setosa' 'Iris-versicolor' 'Iris-setosa' 'Iris-versicolor'
 'Iris-virginica' 'Iris-versicolor' 'Iris-versicolor' 'Iris-versicolor'
 'Iris-virginica' 'Iris-versicolor' 'Iris-setosa' 'Iris-virginica'
 'Iris-setosa' 'Iris-virginica' 'Iris-virginica' 'Iris-virginica'
 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-setosa' 'Iris-virginica'
 'Iris-versicolor']
Accuracy: 0.9211
```

SET 14 EDA + Perceptron vs Logistic Regression

LEARN Q2

Q1 → Exp 1A / 1B / 2 · Q2 → Exp 4 + Exp 3

Datasets: Q1 `Diabetes.csv`, Q2 `IRIS.csv`.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Diabetes.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(768, 9)
```

OUTPUT

```
   Pregnancies  Glucose  BloodPressure  SkinThickness  Insulin   BMI  \
0            6      148             72             35        0  33.6
1            1       85             66             29        0  26.6
2            8      183             64              0        0  23.3
3            1       89             66             23       94  28.1
4            0      137             40             35      168  43.1

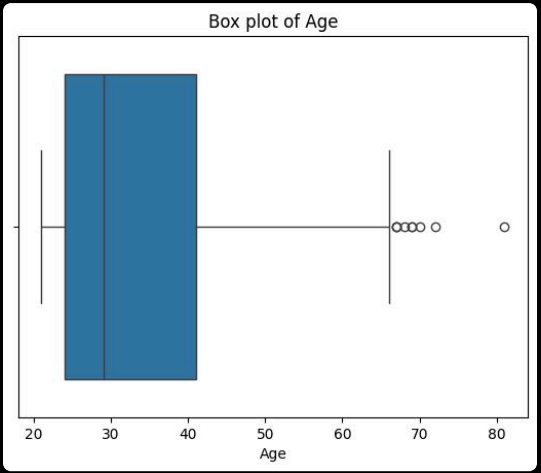
   DiabetesPedigreeFunction  Age  Outcome
0                     0.627   50        1
1                     0.351   31        0
2                     0.672   32        1
3                     0.167   21        0
4                     2.288   33        1
```

Q1.2 5-number summary, correlated with a box plot

```
col = "Age"
print(df[col].describe()[["min", "25%", "50%", "75%", "max"]])     # the 5-number
  summary

sns.boxplot(x=df[col])
plt.title("Box plot of " + col)
plt.show()
```

```
OUTPUT
min    21.0
25%    24.0
50%    29.0
75%    41.0
max    81.0
Name: Age, dtype: float64
```

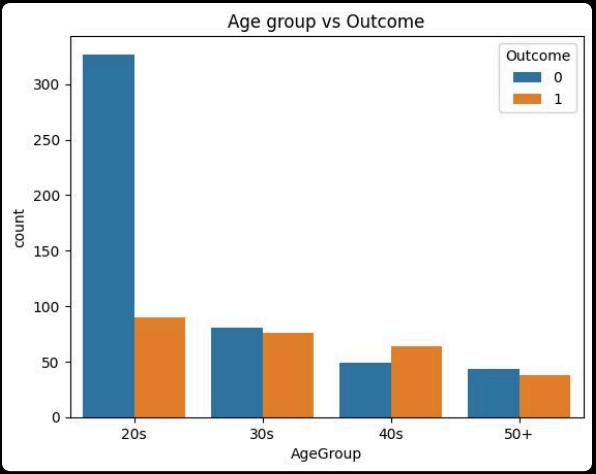


Correlate with the box plot: left edge of the box = 25%, middle line = 50% (median), right edge = 75%, whiskers reach towards min and max, and dots beyond the whiskers are outliers.

Q1.3 Bivariate analysis of two categorical variables (count plot)

Only Outcome is categorical, so make age groups as the second one.

```
df["AgeGroup"] = pd.cut(df["Age"], bins=[20, 30, 40, 50, 90], labels=["20s", "30s",
   "40s", "50+"])
sns.countplot(x="AgeGroup", hue="Outcome", data=df)
plt.title("Age group vs Outcome")
plt.show()
```



Q1.4 Binarise a numerical column with a threshold

```
from sklearn.preprocessing import Binarizer

df["HighGlucose"] = Binarizer(threshold=140).fit_transform(df[["Glucose"]]).ravel()
   # > 140 -> 1, else 0
print(df[["Glucose", "HighGlucose"]].head())
print(df["HighGlucose"].value_counts())
```

```
OUTPUT
   Glucose  HighGlucose
0      148            1
1       85            0
2      183            1
3       89            0
4      137            0
HighGlucose
0    576
1    192
Name: count, dtype: int64
```

Q1.5 Remove highly correlated features

```
num = df.select_dtypes("number").drop(columns=["Outcome", "HighGlucose"])
corr = num.corr().abs()
upper = corr.where(np.triu(np.ones(corr.shape), k=1).astype(bool))    # each pair
   once (upper triangle)
to_drop = [c for c in upper.columns if (upper[c] > 0.5).any()]
print("Highly correlated (> 0.5), dropping:", to_drop)
df = df.drop(columns=to_drop)
```

```
OUTPUT
Highly correlated (> 0.5), dropping: ['Age']
```

> Diabetes features are only weakly correlated, so 0.5 is used here. With strongly correlated data, use 0.8 or 0.9.

Question 2

Q2.1 Read and divide into training (70%) and testing (30%)

IRIS has 3 species. The question wants **2 categories**, so keep versicolor and virginica (the two that overlap).

```
from sklearn.preprocessing import StandardScaler

df = pd.read_csv("data/IRIS.csv")
df = df[df["Species"] != "Iris-setosa"]
X = StandardScaler().fit_transform(df.drop(columns="Species"))
y = df["Species"]

from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3,
   random_state=42, stratify=y)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
Train: (70, 4)  Test: (30, 4)
```

Q2.2–Q2.5 Build both models, evaluate, confusion tables, metrics

```
from sklearn.linear_model import Perceptron, LogisticRegression
from sklearn.metrics import confusion_matrix, accuracy_score, precision_score,
   recall_score, f1_score

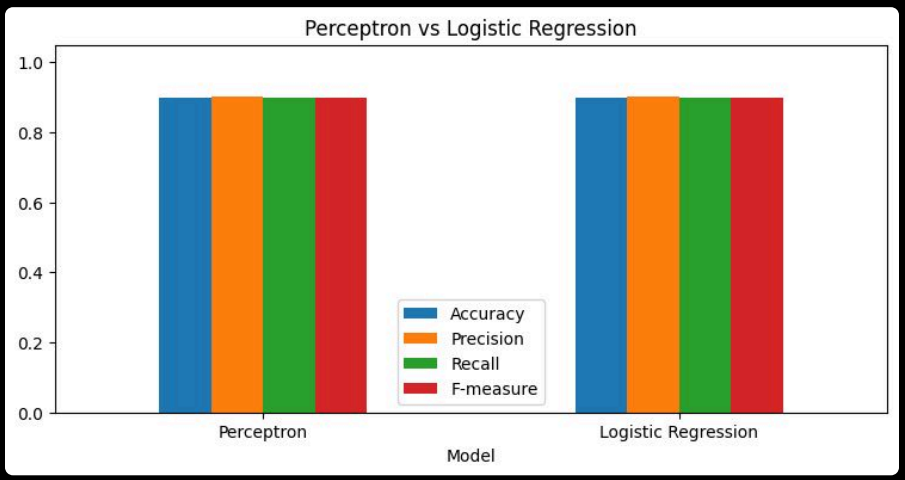
rows = []
for name, model in [("Perceptron", Perceptron(random_state=42)), ("Logistic
   Regression", LogisticRegression())]:
    pred = model.fit(X_train, y_train).predict(X_test)
    print(name, "confusion table:\n", confusion_matrix(y_test, pred))
    rows.append([name, accuracy_score(y_test, pred), precision_score(y_test, pred,
   average="macro"),
                 recall_score(y_test, pred, average="macro"), f1_score(y_test,
   pred, average="macro")])
results = pd.DataFrame(rows, columns=["Model", "Accuracy", "Precision", "Recall",
   "F-measure"]).set_index("Model")
results.round(4)
```

```
OUTPUT
Perceptron confusion table:
 [[14  1]
 [ 2 13]]
Logistic Regression confusion table:
 [[14  1]
 [ 2 13]]
```

```
OUTPUT
                    Accuracy  Precision  Recall  F-measure
Model
Perceptron               0.9     0.9018     0.9     0.8999
Logistic Regression      0.9     0.9018     0.9     0.8999
```

Q2.6 Plot the results and compare

```
results.plot(kind="bar", rot=0, figsize=(9, 4))
plt.ylim(0, 1.05)
plt.title("Perceptron vs Logistic Regression")
plt.show()
```



SET 15 EDA + MLP Regression with Randomized Search

LEARN Q2

Q1 → Exp 1A / 1B / 2 · Q2 → Exp 5 (MLP regression)

Datasets: Q1 `Solar.csv`. Q2 asks for `Solar-regression.csv`, which we don't have, so this uses `Kasol.csv`, predicting its continuous `Solar` column (the same stand-in as Set 5).

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import warnings
warnings.filterwarnings("ignore")      # hides library warnings so the output stays
  clean
```

Question 1

Q1.1 Read the data

```
df = pd.read_csv("data/Solar.csv")
print(df.shape)
df.head()
```

OUTPUT

```
(228, 7)
```

OUTPUT

```
   Latitude  Longitude  Altitude  min Temo  Max Temp  Sunshine Hour  \
0     34.08      74.79      1730      -0.9       4.3       0.450000
1     34.08      74.79      1730      -0.2       7.6       0.464286
2     34.08      74.79      1730       2.6      14.2       0.521429
3     34.08      74.79      1730       1.8       8.1       0.542857
4     34.08      74.79      1730       7.5      20.4       0.628571

   Solar Radiation
0                0
1                0
2                0
3                0
4                0
```

Q1.2 Identify the numerical columns: min, max and mean

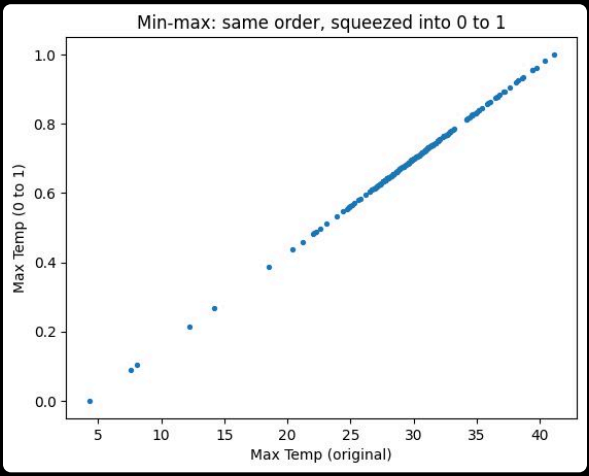
```
num = df.select_dtypes("number")
print("Numerical columns:", list(num.columns))
pd.DataFrame({"min": num.min(), "max": num.max(), "mean": num.mean().round(2)})
```

```
OUTPUT
Numerical columns: ['Latitude', 'Longitude', 'Altitude', 'min Temo', 'Max Temp',
'Sunshine Hour', 'Solar Radiation']
```

```
OUTPUT
                      min      max    mean
Latitude         8.280000    34.08   20.02
Longitude       72.150000    92.72   78.86
Altitude         2.000000  1730.00  320.00
min Temo        -0.900000    29.70   21.95
Max Temp         4.300000    41.10   29.82
Sunshine Hour    0.428571    11.00    1.73
Solar Radiation  0.000000     1.00    0.50
```

Q1.3 Min-max normalisation of a numerical feature, analysed with a scatter plot

```
col = "Max Temp"
df[col + "_minmax"] = (df[col] - df[col].min()) / (df[col].max() - df[col].min())
plt.scatter(df[col], df[col + "_minmax"], s=8)
plt.xlabel(col + " (original)")
plt.ylabel(col + " (0 to 1)")
plt.title("Min-max: same order, squeezed into 0 to 1")
plt.show()
```



Q1.4 Label encoding of a categorical feature

Solar.csv has no text column, so make one (temperature level) and encode it.

```
df["TempLevel"] = pd.cut(df["Max Temp"], bins=3, labels=["Low", "Medium", "High"])
from sklearn.preprocessing import LabelEncoder

print("Before:", df["TempLevel"].unique()[:6])
df["TempLevel"] = LabelEncoder().fit_transform(df["TempLevel"].astype(str))
print("After :", df["TempLevel"].unique()[:6])
```

```
OUTPUT
Before: ['Low', 'Medium', 'High']
Categories (3, object): ['Low' < 'Medium' < 'High']
After : [1 2 0]
```

Q1.5 Remove features with low variance

```python
from sklearn.feature_selection import VarianceThreshold

num = df.select_dtypes("number").drop(columns=["Max Temp_minmax", "TempLevel"])   # only the original columns
print(num.var().round(3))
vt = VarianceThreshold(threshold=0.3).fit(num)
print("\nKept   :", list(num.columns[vt.get_support()]))
print("Removed:", list(num.columns[~vt.get_support()]))
```

```
OUTPUT
Latitude              36.801
Longitude             33.851
Altitude          184001.656
min Temo              32.802
Max Temp              23.253
Sunshine Hour          6.958
Solar Radiation        0.251
dtype: float64

Kept   : ['Latitude', 'Longitude', 'Altitude', 'min Temo', 'Max Temp', 'Sunshine Hour']
Removed: ['Solar Radiation']
```

Question 2

Q2.1 Read and divide into training (75%) and testing (25%)

```python
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split

data = pd.read_csv("data/Kasol.csv").dropna()                # drop the empty lines (and 14 rows with missing Rainfall)
X = StandardScaler().fit_transform(data.drop(columns=["Unnamed: 0", "Solar"]))   # drop the date and the target
y = data["Solar"]
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25, random_state=42)
print("Train:", X_train.shape, " Test:", X_test.shape)
```

```
OUTPUT
Train: (5468, 6)  Test: (1823, 6)
```

Q2.2 Build the prediction model

Q2.3 Evaluate on the test set: MSE and RMSE

One code cell below answers both parts.

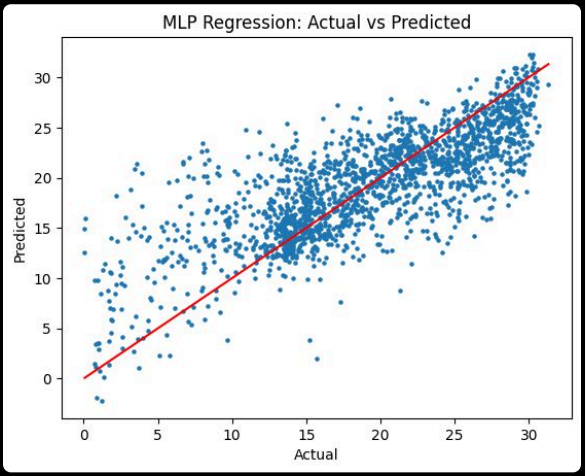
```
from sklearn.neural_network import MLPRegressor
from sklearn.metrics import mean_squared_error

model = MLPRegressor(hidden_layer_sizes=(32,), max_iter=300, early_stopping=True,
   random_state=42).fit(X_train, y_train)
y_pred = model.predict(X_test)
mse = mean_squared_error(y_test, y_pred)
print("MSE :", round(mse, 4))
print("RMSE:", round(np.sqrt(mse), 4))
```

```
OUTPUT
MSE : 18.9464
RMSE: 4.3527
```

Q2.4 Plot the actual and predicted values

```
plt.scatter(y_test, y_pred, s=5)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], color="red")
plt.xlabel("Actual")
plt.ylabel("Predicted")
plt.title("MLP Regression: Actual vs Predicted")
plt.show()
```



Q2.5 Improve with Randomized Search CV and display the optimal hyperparameters

```
from sklearn.model_selection import RandomizedSearchCV

params = {"hidden_layer_sizes": [(32,), (64,), (64, 32)], "alpha": [0.0001, 0.001,
   0.01], "learning_rate_init": [0.001, 0.01]}
rand = RandomizedSearchCV(MLPRegressor(max_iter=300, early_stopping=True,
   random_state=42), params,
                          n_iter=5, cv=3, random_state=42).fit(X_train, y_train)
print("Optimal hyperparameters (Randomized):", rand.best_params_)
rmse_rand = np.sqrt(mean_squared_error(y_test, rand.predict(X_test)))
print("RMSE after Randomized Search:", round(rmse_rand, 4))
```

```
OUTPUT
Optimal hyperparameters (Randomized): {'learning_rate_init': 0.01, 'hidden_layer_sizes':
(64, 32), 'alpha': 0.0001}
RMSE after Randomized Search: 4.3519
```

Q2.6 Compare the results

```
rmse = {"MLP (default)": np.sqrt(mse), "MLP (Randomized Search)": rmse_rand}
print(rmse)
plt.bar(rmse.keys(), rmse.values())
plt.ylabel("RMSE (lower = better)")
plt.title("Before vs after Randomized Search")
plt.show()
```

```
OUTPUT
{'MLP (default)': np.float64(4.352747101352028), 'MLP (Randomized Search)':
np.float64(4.351851242445318)}
```

